AKADEMIA GÓRNICZO-HUTNICZA IM. STANISŁAWA STASZICA W KRAKOWIE
WYDZIAŁ ELEKTROTECHNIKI, AUTOMATYKI, INFORMATYKI I INŻYNIERII BIOMEDYCZNEJ

Praca dyplomowa

Hybrid Multi-Agent Trading System Integrating Heterogeneous AI Methods

Hybrydowy Wieloagentowy System Transakcyjny Integrujący Zróżnicowane Techniki Sztucznej Inteligencji

Autor: Wojciech Neuman
Kierunek studiów: Informatyka i Systemy Inteligentne
Opiekun pracy: dr inż. Krzysztof Kluza

Kraków, 2025

ABSTRACT

This thesis develops and evaluates a hybrid multi-agent system for algorithmic trading, in which each agent relies on a distinct artificial-intelligence technique. It draws on OHLCV price data, technical indicators, microstructure and market-structure features, and external sentiment and cross-asset signals, and is trained and evaluated on hourly Bitcoin (BTC/USDT) data under a strict, leakage-controlled walk-forward protocol.

The final system is a fund of seven autonomous agents: four learned models from different paradigms — a gradient-boosting model (LightGBM), a temporal convolutional network (TCN), a patch-based Transformer (PatchTST), and a selective state-space model (Mamba) — and three rule-based agents (trend following, volatility breakout, and cross-asset dominance rotation). Each produces its own risk-managed stream of returns, combined by a leak-free, capped inverse-volatility allocator. These four learned models are the ones that came through a broad search over state-of-the-art techniques with strong out-of-sample results; other approaches explored along the way — deep reinforcement learning, evolutionary rule generation, and recurrent networks — did not, and are left out of the final system.

Over a two-year out-of-sample window spanning three market regimes, the diversified fund achieved the strongest *risk-adjusted* performance in the study: a +49.7% return at a Sharpe ratio of 1.92 and a maximum drawdown of only −5.3%, profitable in every regime and well ahead of Bitcoin buy-and-hold (Sharpe 0.08) and the S&P 500 (Sharpe 1.16). A learned adaptive coordinator failed to beat the far simpler risk-parity rule, and the highest-returning single agent did not survive a random-bracket test for genuine skill. The central conclusion is that combining heterogeneous AI methods *can* yield more stable, regime-balanced performance than any single approach, with the value lying in disciplined diversification across genuinely diverse agents rather than in a sophisticated controller. The strong out-of-sample results make an encouraging case for the approach, and live deployment — with its real-world execution costs — is the natural next step.

Contents

1 Introduction

Financial markets are among the largest and most consequential time-series systems ever created. Every second, currencies, equities, commodities, and cryptocurrencies generate an enormous stream of prices and trades, and the cost of a poor decision is immediate and quantifiable. It is no surprise, then, that the question of whether future prices can be anticipated — even partially — has occupied investors, economists, and, increasingly, computer scientists for more than a century. With the arrival of machine learning and, more recently, deep learning, this old question has been given new tools, and a substantial body of research now studies financial forecasting as a problem of pattern recognition under extreme noise.

There are many ways to interact with a market, and they form a continuum rather than a set of discrete categories. One can invest for decades, holding an asset across entire business cycles in the belief that its underlying value will grow; one can position over months, rotating between sectors as the macroeconomic backdrop shifts; or one can trade on hourly — or even sub-minute — horizons, seeking to profit from short-lived dislocations. The horizon one chooses dictates almost everything else: the data, the methods, and the very notion of what counts as a signal.

Two broad analytical traditions have historically guided these decisions. *Fundamental analysis* studies the economic standing of an asset. For a company this means earnings, debt levels, cash flow, and competitive position. For a cryptocurrency it means adoption trends, on-chain network activity, and the rules governing its monetary supply. Because fundamentals change slowly and are expressed in unstructured form, fundamental analysis is inherently long-horizon and difficult to automate [1]. *Technical analysis*, by contrast, derives signals directly from price and volume through indicators such as moving averages,

the Relative Strength Index (RSI)[1], and the Moving Average Convergence Divergence (MACD). These operate on whatever timeframe they are fed and are therefore the natural language of short- and medium-horizon trading. This thesis works squarely in the technical, data-driven tradition, at a swing-trading horizon of hours to days, where the volume of data is large enough to train modern models and the feedback is fast enough to evaluate them.

A persistent finding across the forecasting literature is that *no single method is consistently best*. Deep sequence models capture rich temporal structure, but they tend to break down when market conditions shift in ways not seen during training. Gradient-boosted trees handle tabular features well yet are blind to sequential patterns. Rule-based strategies are easy to interpret but too rigid to adapt. Reinforcement-learning agents optimise returns directly, yet they routinely overfit to the simulated environments they were trained in. Each paradigm sees only part of the picture, and each fails in its own characteristic way. This observation is the intellectual starting point of the present work.

Rather than searching for a single best model, this thesis asks whether *combining structurally different methods* can produce more stable and reliable trading decisions than any of them alone. We use two terms deliberately throughout. A system is *heterogeneous* when its components are built from categorically different learning paradigms — not variations of one technique but distinct families, each with its own inductive bias and failure mode. A system is *hybrid* when these dissimilar components are integrated into a single decision-making whole. The central hypothesis is that heterogeneity, properly combined, is a source of robustness: when several agents fail in uncorrelated ways, the system as a whole can stay steady where any one of them alone would struggle.

The application domain is the cryptocurrency market, and specifically Bitcoin (BTC). Bitcoin trades continuously without session gaps, is highly liquid and volatile, carries fewer of the regulatory and fundamental constraints that complicate equities, and is widely studied, which makes it an ideal testbed. As we argue later, forecasting Bitcoin alone is

[1] Acronyms are expanded at first use and collected, with short plain-language definitions, in the Glossary at the end of the thesis.

already hard enough to occupy an entire thesis; the broader cryptocurrency universe enters only indirectly, as a way to approximate market-wide capital flows.

The remainder of the thesis is organised as follows. Chapter 2 establishes the theoretical background: the statistical character of financial time series, the principles of feature engineering, the artificial-intelligence methods relevant to trading, and the notion of an agent and a multi-agent system. Chapter 3 presents the proposed architecture — the heterogeneous base agents, the rule-based agents, and the capital-allocation layer that binds them into a fund. Chapter 4 details the data, the feature pipeline and its leakage controls, and the construction and training of every agent. Chapter 5 describes the software and reproducible pipeline. Chapter 6 reports the experimental results, both for the individual agents and for the integrated system, including the honest negative results. Chapter 7 formalises the financial evaluation and transaction-cost model, and Chapter 8 interprets the findings and their limitations. Chapter 9 concludes the thesis and sets out directions for future work.

2 Theoretical Background

Before presenting the proposed system, it is necessary to introduce the theoretical background related to financial time series and their analysis using artificial intelligence methods. This thesis focuses on the application of AI techniques to financial data, which requires understanding both the properties of time series and the models used to process them. The following subsections describe the main characteristics of financial time series, the role of feature engineering, and selected artificial intelligence methods used in algorithmic trading. The chapter also introduces ensemble methods and decision fusion, as well as the concept of agents and multi-agent systems, which form the basis of the proposed approach.

Financial Markets and Time Series Characteristics

A time series is a collection of observations indexed by the date of each observation [2]. Most commonly, it is a sequence of values recorded at successive, equally spaced points in time, forming discrete-time data [3]. Examples of such time series include environmental readings (e.g., temperature, rainfall), business data (e.g., sales, web traffic), and financial metrics such as asset prices. Time series are analyzed to uncover patterns, trends, and relationships that evolve over time, enabling informed decisions and predictions [4]. In financial markets, observations of an asset's price taken at equally spaced time intervals constitute a financial time series [4]. These observations are typically represented in the form of OHLCV data, which includes the open, high, low, and close prices, as well as the traded volume within a given time interval. Depending on the chosen timeframe, such data can be recorded at different resolutions, ranging from high-frequency (e.g., tick or minute-level) to lower-frequency intervals such as daily or weekly data. Beyond aggregated

price data, financial markets also generate more detailed information, such as limit order book (LOB) data, which captures the supply and demand dynamics of the market. While OHLCV data provides a compact representation of price movements, LOB data offers a more granular view of market activity and can be used to derive additional features for analysis [5]. Financial time series exhibit several properties that make modeling and prediction challenging. Returns typically show weak autocorrelation, meaning that past price changes provide limited direct information about future movements, while volatility measures, such as absolute or squared returns, often exhibit significant persistence. In addition, financial data is non-stationary, with statistical properties such as mean and variance changing over time, and displays volatility clustering, where periods of high variability tend to persist. Return distributions are also heavy-tailed, meaning that extreme events occur more frequently than expected under normal assumptions. Finally, nonlinear dependencies and long memory effects further complicate modeling, motivating the use of more advanced data-driven approaches [6]. These characteristics highlight the complexity of financial time series and the challenges associated with their modeling, which require careful feature design and appropriate modeling approaches.

Feature Engineering in Financial Data

Raw OHLCV data alone is rarely sufficient as direct input to a predictive model. Feature engineering turns this raw signal into a richer representation, bringing out the structure, momentum, and volatility that otherwise stay buried in the bare price and volume series [7].

Technical indicators. The most widely used features in algorithmic trading are technical indicators computed directly from OHLCV data. *Trend* indicators such as Simple Moving Averages (SMA) and Exponential Moving Averages (EMA) smooth the price series and express the current price relative to its recent history; expressing close price as a ratio to, for example, the 200-period SMA separates the long-run trend from short-run fluctuations. *Momentum* indicators capture the rate and direction of recent price changes: the Relative Strength Index (RSI) normalises average gains and losses over a

rolling window to produce an oscillator bounded between 0 and 100; the Moving Average Convergence Divergence (MACD) histogram measures the difference between a fast and a slow EMA, highlighting shifts in momentum. *Volatility* indicators such as the Average True Range (ATR) and Bollinger Bands quantify the range of price variation, which is essential both for risk management (setting stop-loss distances) and as a regime signal (periods of low volatility often precede explosive moves).

Volume and microstructure features. Volume confirms or contradicts price movement: a breakout accompanied by high volume is a qualitatively different signal from the same breakout on thin volume. Volume-weighted indicators such as the Volume-Weighted Average Price (VWAP) and the Accumulation/Distribution line explicitly combine price and volume. At higher granularity, microstructure features derived from Binance's taker-volume feed — including the Taker Flow Imbalance (TFI), average trade size, and taker price premium — provide direct evidence of aggressive buying and selling pressure, which is otherwise invisible in aggregated OHLCV candles.

Statistical and non-stationarity features. Financial returns are non-stationary: the unconditional mean and variance of log-prices drift over time, which breaks the assumptions of most machine learning methods, since those methods expect independent, identically distributed data. Several approaches address this: computing log-returns or returns at fixed lags removes the deterministic trend; the Hurst exponent, estimated over a rolling window, distinguishes trending regimes ($H > 0.5$) from mean-reverting ones ($H < 0.5$) [6]. Rolling skewness, kurtosis, and autocorrelation coefficients describe the local return distribution and are useful inputs for volatility-regime models.

Fractional differentiation. A principled approach to balancing stationarity and memory preservation is fractional differentiation, introduced by Hosking [8] and applied to financial machine learning by López de Prado [9]. Integer differentiation (i.e., computing first differences) achieves stationarity but discards all long-memory content; for a log-price series, this means losing the level information that market participants actually respond to. The fractional difference operator of order $d \in (0, 1)$ applies an infinite-order moving average with weights that decay hyperbolically, achieving stationarity for sufficiently

large d while retaining long-range dependencies. In practice, the minimum d for which an Augmented Dickey-Fuller test rejects the unit-root null hypothesis is found by bisection search, and the resulting series is used as an additional input feature alongside the integer-differenced returns.

Labelling methodology: Triple Barrier Method. Defining the prediction target is as important as feature construction. Naive next-bar labelling — classifying bars as *up* if $\text{close}_{t+1} > \text{close}_t$ — assigns equal weight to trivially small moves and large directional moves, producing noisy labels dominated by the bid–ask spread and microstructure noise [9]. The Triple Barrier Method (TBM), introduced by López de Prado [9], resolves this by placing three barriers around each entry point: an upper horizontal barrier at $+k\sigma$, a lower horizontal barrier at $-k\sigma$ (where σ is a rolling estimate of return volatility), and a vertical barrier after $T_{\max}$ bars. Each bar is labelled by whichever barrier is touched first: label 1 (up) if the upper barrier is reached, label 0 (down) if the lower barrier is reached, and label 2 (flat) if the vertical barrier expires before either horizontal barrier is hit. The parameter k controls the signal-to-noise ratio of the resulting labels: tighter barriers produce more samples but noisier ones; wider barriers produce fewer, cleaner labels with stronger directional content.

Multi-timeframe and calendar features. Market participants operate on different horizons simultaneously. Features that capture multi-timeframe alignment — for instance, whether the 4-hour RSI, the daily trend, and the weekly momentum all point in the same direction — encode higher-level structure that single-timeframe indicators miss. Calendar features (hour of day, day of week, encoded as sine/cosine pairs to preserve periodicity) capture intraday and intraweek seasonality. For Bitcoin specifically, the approximately four-year halving cycle introduces macro-seasonal dynamics that can be encoded as a cycle-phase variable.

Artificial Intelligence Methods in Algorithmic Trading

The growing availability of financial data and advances in computational power have driven the widespread adoption of artificial intelligence methods in algorithmic trading.

These methods range from classical statistical approaches to modern deep learning architectures, each capturing different aspects of market dynamics. Sezer et al. [10] surveyed 140 deep learning studies published between 2005 and 2019 and found that LSTM and its variants dominated as the architecture of choice, with stock price forecasting, trend forecasting, and index forecasting together accounting for more than 70% of all publications. Zhang et al. [4] extended this picture to 2022, confirming the continued prominence of recurrent architectures while noting growing interest in Transformer-based models. Sonkavde et al. [7] provide a broader review encompassing both classical and deep learning methods alongside traditional statistical techniques.

Classical and statistical approaches. Before machine learning took hold, automated trading ran mostly on hand-coded rules — moving-average crossovers, momentum thresholds, and the like — that captured a trader's expertise directly. Such rules are easy to read and cheap to run, but they do not adapt when the market changes [7]. Statistical models such as ARIMA and its extensions capture linear temporal dependencies and serve as theoretically grounded baselines, but their assumption of linearity and stationarity limits their effectiveness on financial data where nonlinear dynamics and regime changes are the norm [4, 7].

Machine learning methods. Machine learning methods learn patterns directly from data without requiring a predefined functional form. Tree-based ensembles — most notably Random Forest and XGBoost — have demonstrated competitive performance across multiple benchmarks. Sonkavde et al. [7] showed that combining such models with LSTM outperformed any individual model, illustrating the benefit of heterogeneous ensembles. Support vector machines and k-nearest neighbours are also widely used, though they scale poorly to high-dimensional feature spaces without careful preprocessing.

LightGBM. LightGBM [11] is a gradient boosting framework that builds an additive ensemble of decision trees by fitting each new tree to the residuals of all previous ones. Its key technical contributions relative to earlier gradient boosting implementations are (i) *leaf-wise* (best-first) tree growth, which preferentially expands the leaf with the largest

loss reduction rather than growing all leaves at the same depth level; and (ii) *histogram-based splitting*, which discretises continuous feature values into bins, dramatically reducing the cost of finding optimal splits. Together these innovations allow LightGBM to train on large datasets with many features at a fraction of the cost of XGBoost while matching or exceeding its predictive accuracy. For financial data, the combination of efficient handling of high-dimensional tabular input, native support for early stopping on a held-out validation set, and built-in feature importance scores makes LightGBM a natural choice for the supervised classification component of the system.

Deep learning for time series. Deep learning has become the dominant paradigm for financial forecasting due to its ability to learn hierarchical representations without heavy reliance on manual feature design [4]. LSTM networks address the vanishing gradient problem of plain RNNs through gating mechanisms that selectively retain or discard information over long sequences, making them the most widely applied architecture in the reviewed literature [10, 4]. The GRU offers a lighter alternative with comparable accuracy. Transformer models leverage self-attention to capture long-range dependencies without sequential computation; efficient variants such as Informer, Autoformer, and PatchTST have specifically addressed the scalability challenges of applying Transformers to long financial sequences [4].

Temporal Convolutional Networks. Temporal Convolutional Networks (TCNs), introduced as a general sequence modelling architecture by Bai et al. [12], build on the *dilated causal convolution* mechanism originally proposed for audio generation in WaveNet [13]. A causal convolution restricts each output to depend only on past and current inputs, preserving the temporal ordering required for financial forecasting without data leakage. Dilation exponentially increases the receptive field of deeper layers: with a kernel of size k and dilation factor d, a single convolutional layer covers a span of $(k-1) \times d + 1$ timesteps. A stack of L blocks with dilations $1, 2, 4, \ldots, 2^{L-1}$ achieves a total receptive field of $1 + 2(k-1)(2^L - 1)$, which grows exponentially with depth rather than linearly as in standard convolutions. Each TCN block pairs two dilated causal convolutions with weight normalisation [14] and dropout regularisation, followed by a residual connection

that projects the input to the same width as the output using a 1×1 convolution when necessary. Unlike recurrent architectures, TCNs process the entire input sequence in parallel during training, leading to faster convergence and more stable gradients. Bai et al. [12] demonstrated that this simple architecture matches or outperforms LSTM and GRU across a broad range of sequence modelling benchmarks while being significantly easier to train.

Multi-task learning. Multi-task learning trains a shared representation to simultaneously optimise two or more related objectives [4]. In the context of financial forecasting, a natural auxiliary task alongside directional classification is volatility forecasting: both tasks are grounded in the same underlying market dynamics, so gradients from the auxiliary regression task regularise the shared encoder and steer it away from fitting noise in the classification labels. The combined loss is typically a weighted sum $\mathcal{L} = \mathcal{L}_{\text{class}} + \lambda \mathcal{L}_{\text{aux}}$, where λ controls the strength of the auxiliary signal.

Reinforcement learning. Reinforcement learning frames trading as a sequential decision problem in which an agent selects actions — buy, sell, or hold — and receives rewards reflecting realised returns. Unlike supervised methods, RL agents directly optimise cumulative trading performance, making the framework naturally aligned with the profit-maximisation objective. Value-based approaches such as Deep Q-Networks and policy gradient methods such as Proximal Policy Optimisation are the most common formulations. Sezer et al. [10] identify deep RL as one of the most promising directions for future research, particularly for applications requiring continuous adaptation to changing market conditions.

Evolutionary computation. Evolutionary methods such as genetic algorithms and neuroevolution provide gradient-free optimisation applicable to strategy design, feature selection, and hyperparameter tuning. In the financial deep learning literature they have been used primarily as auxiliary optimisers rather than standalone strategy generators [7, 10], though neuroevolution approaches additionally offer the ability to evolve network architectures alongside weights when gradient information is unavailable or unreliable.

Ensemble Methods and Decision Fusion

Ensemble methods improve predictive performance by combining multiple models whose errors are at least partially uncorrelated. The two classical approaches are *bagging* (training the same model on different bootstrap samples of the data, reducing variance) and *boosting* (training models sequentially, each correcting the residuals of its predecessors, reducing bias). Gradient boosting, as implemented in LightGBM, is the most prominent boosting framework in tabular machine learning.

Stacking and *meta-learning* generalise this idea: predictions from K base models are used as features for a second-level model (the meta-learner), which learns how to weight or combine them conditional on the input. When the base models are heterogeneous — using different architectures and different views of the data — stacking can substantially outperform any individual component, because the meta-learner can learn when to trust each source. Sonkavde et al. [7] provide empirical evidence for this on financial data, finding that LSTM combined with gradient boosting outperformed either model alone. This observation directly motivates the hybrid multi-agent architecture of this thesis.

Multi-Agent Systems

An *agent* is a computer system situated in an environment that is capable of autonomous action in order to meet its design objectives [15]. More formally, an agent exhibits at least the following properties:

- **autonomy**: operates without direct human intervention and has control over its actions and internal state;
- **social ability**: interacts with other agents (and possibly humans) via a communication mechanism;
- **reactivity**: perceives its environment and responds to changes in a timely manner;
- **pro-activeness**: exhibits goal-directed behaviour by taking initiative, not only reacting to the environment.

A *multi-agent system* (MAS) consists of a network of such agents that interact to solve problems that are beyond the capacity or knowledge of any individual agent. MAS architectures are commonly classified as centralised (a single coordinating agent with global visibility), decentralised (agents act independently, with coordination emerging from local interactions), or hierarchical (agents are organised into layers, with higher-level agents aggregating signals from lower-level ones).

While the concept of intelligent agents dates back to the early 1990s [15], multi-agent systems have more recently gained renewed attention due to the rapid proliferation of Large Language Models (LLMs), which act as flexible reasoning engines capable of delegating tasks, using tools, and collaborating with other agents. This thesis focuses on the classical, signal-aggregation interpretation of MAS rather than LLM-based orchestration: each agent implements a distinct AI method and produces a trading signal, which a supervisory agent then combines into a final decision. The key design principle is *heterogeneity*: by coupling models with structurally different inductive biases — a gradient boosting agent that excels at tabular feature selection, sequence models that capture temporal patterns, and rule-based agents that read economic structure directly — the system can exploit complementary sources of edge while diversifying away method-specific weaknesses.

3 System Architecture

3.1 Overall Architecture

The proposed system is a *heterogeneous and hybrid* multi-agent architecture, best understood as a small *fund of autonomous trading agents*. The term *heterogeneous* refers to the nature of the constituent components: the agents are built from fundamentally different paradigms — gradient boosting, dilated causal convolutions, selective state-space modelling, patch-based transformer attention, and classical rule-based trading logic. They are not variations of a single technique but categorically distinct methods, each with its own inductive bias and failure mode. The term *hybrid* refers to the architecture — the fact that these dissimilar components are integrated into a unified system that produces a single equity curve. Integration is achieved not by a single learned controller but by a capital-allocation layer that distributes risk across the agents.

Seven agents are retained in the final system: four *learned* agents — LightGBM, a Temporal Convolutional Network (TCN), a selective state-space model (Mamba), and a patch-based Transformer (PatchTST) — and three *rule-based* agents — a trend follower, a volatility-breakout strategy, and a cross-asset dominance-rotation strategy. Each agent is fully autonomous: it carries its own feature subset, its own directional signal, and its own risk-managed execution rule, and it produces an independent stream of positions, confidence values, and realised returns. This autonomy is what makes the system a fund rather than a single model with auxiliary inputs.

Figure 3.1 illustrates the high-level data flow. Raw market data is ingested and processed into layered feature libraries. Each agent selects a subset of these libraries according to its own constraints and operates an independent walk-forward training and inference

loop, producing an out-of-sample (OOS) return stream. Every stream is evaluated by the same backtest, which brackets each trade with a stop-loss and take-profit set from recent volatility — the Average True Range (ATR) — and charges realistic fees; the mechanism is detailed in Chapter 4. The streams are then aligned on a common index and combined by a leak-free, risk-balanced allocator into the final fund equity curve. A learned regime-gated coordinator was also built, but it did not improve on this simpler allocator and is reported only as a comparison.

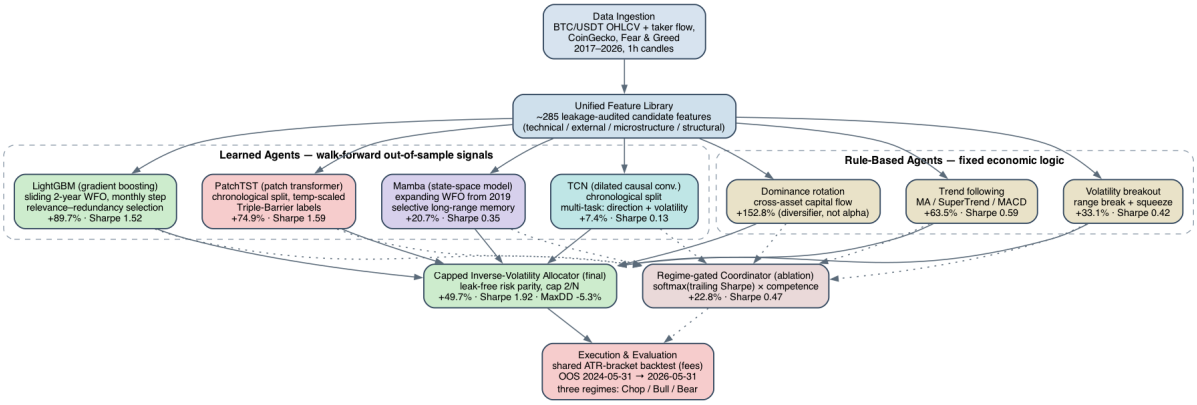

Figure 3.1: High-level architecture of the hybrid multi-agent trading system. Raw OHLCV data and external feeds are processed into a unified feature library spanning four feature families (technical, external, microstructure, structural). Seven agents — four learned (LightGBM, TCN, Mamba, PatchTST) and three rule-based (trend, volatility breakout, dominance rotation) — each consume a distinct feature subset and produce an independent, risk-managed out-of-sample return stream through a shared ATR-bracket backtest. A leak-free capped inverse-volatility allocator combines these streams into the final fund equity curve; a learned regime-gated coordinator is retained as an ablation. All evaluation spans the three market regimes of the two-year hold-out window.

3.2 The Learned Agents

The system's four learned agents are deliberately drawn from four different families, so that each carries a distinct inductive bias. This section introduces their *roles*; the full specification of every agent — its feature diet, labelling, architecture, and training protocol — is given in Chapter 4.

LightGBM is the tabular agent. Gradient-boosted decision trees consume a compact feature subset selected from the full candidate pool and emit a directional probability at every hourly bar. Treating each bar as an independent feature vector, it excels at

exploiting shallow non-linear interactions in the high-dimensional technical panel, and it is the most reliable single performer in the system.

The Temporal Convolutional Network (TCN) is the local-sequence agent. Stacked dilated causal convolutions read a multi-day window of features and learn temporal patterns — momentum build-up, volatility clustering, multi-bar candle formations — that the tabular view cannot see. It is trained multi-task, forecasting both direction and forward volatility, and labelled with the Triple Barrier Method.

The Mamba agent is the selective-memory agent. Its state-space architecture adapts its transitions to the input at each step, choosing what historical context to retain or forget, which is well matched to a market that shifts between regimes. It complements the TCN's fixed receptive field with input-dependent long-range memory.

The PatchTST agent is the attention agent. It splits the input window into coarse patches and applies self-attention across them, relating non-adjacent market segments that a convolution cannot connect directly. It shares the TCN's feature diet, so the two agents differ by architecture alone, and its calibrated probabilities make it a selective, low-turnover contributor.

3.3 The Rule-Based Agents

The four learned agents are all non-linear models fitted to overlapping views of the same price-and-technical feature panel, so they tend to share failure modes. To inject genuinely different sources of edge, three *rule-based* agents are added, whose signals come from fixed economic logic rather than from fitting the data. Each emits a conviction in $[0, 1]$ (with 0.5 neutral) and is executed through the same ATR-bracket engine as the learned agents.

The trend agent follows direction: it goes long when the moving-average, SuperTrend, and MACD structure is bullish and short when it is bearish. By design it earns in directional (bull or bear) regimes and gives back ground in choppy ones.

The volatility-breakout agent trades the direction of a confirmed range break, with conviction reinforced when the break follows a volatility squeeze. Its edge is partly struc-

tural: the asymmetric take-profit/stop bracket gives it a convex pay-off when volatility expands.

The dominance-rotation agent reads where capital is flowing within the crypto market. It leans long Bitcoin when BTC dominance rises while the ETH/BTC cross weakens (capital rotating into BTC), and flat or short when the alt-season rotation reverses. Its inputs are dominance and cross-asset momentum only, so it reads an information source no price-feature model uses; it is included for this diversification value rather than as a proven source of return.

3.4 Capital Allocation Layer

The integration layer treats each agent as an autonomous sub-strategy that contributes its own stream of returns, and its job is to decide how much capital to give each one. The rule used in the final system is deliberately simple and transparent: give more weight to the agents whose returns have been calmest recently and less to the volatile ones, so that every agent contributes a similar amount of *risk* rather than a similar amount of money. (This idea has a standard name, *inverse-volatility risk parity*.) To stop any single agent from taking over the fund during a lucky streak, each agent's weight is also capped — here at no more than about 29% of the fund. Every weight is computed only from past returns, so the allocation never looks into the future. The precise formula is given in Chapter 4.

For comparison, a more elaborate *regime-gated coordinator* was also built. It is the kind of adaptive controller one might expect to be the centrepiece of an "intelligent" system: it leans on whichever agents have performed best recently and which have historically suited the current type of market. It is kept only as an *ablation* — a controlled comparison in which a fancier component is swapped in for a simpler one to test whether the extra complexity actually helps. Here it does not: the simple capped rule performs better, the clearest evidence that this thesis's contribution is disciplined diversification rather than a clever controller. A naive equal-weight fund (every agent given the same share) is reported throughout as a further baseline. These comparisons are presented in Chapter 6.

3.5 Walk-Forward Evaluation Protocol

All agents that involve trained models are evaluated under a strict walk-forward protocol to prevent any form of look-ahead bias. The LightGBM agent uses a sliding two-year window that steps forward one month at a time, so the model always reflects recent market character; the Mamba agent uses an expanding window that grows from the start of the data and retrains every three months. In both cases each fold is trained only on data preceding the slice it predicts. The TCN and PatchTST agents use a single chronological three-way split (train / validation / test) with the test set reserved exclusively for final evaluation. The rule-based agents are not fitted to the data at all; only their ATR-bracket execution parameters are grid-searched on a pre-OOS validation window (2022–2024) and frozen before the hold-out is touched. The capital-allocation layer is itself causal: every weight is computed from trailing return statistics only, so no future information enters the fund.

This protocol means that every point in the final OOS equity curve is genuinely out-of-sample: the agent that generated it, and the weight it was given, were determined entirely from data preceding the bar in question. The unified hold-out window runs from 31 May 2024 to 31 May 2026.

4 Methodology

4.1 Data

The dataset used in this research consists of historical price data for ten cryptocurrency assets, selected based on their historical peak market capitalisation. All assets are quoted against USDT and sourced exclusively from the Binance exchange via its public REST API, ensuring consistency across symbols in terms of price discovery, liquidity, and data format. Stablecoins were excluded from the selection. The chosen assets are: Bitcoin (BTC), Ethereum (ETH), Binance Coin (BNB), XRP, Solana (SOL), Cardano (ADA), Dogecoin (DOGE), Avalanche (AVAX), Polkadot (DOT), and Chainlink (LINK).

Data collection was performed using a custom paginated downloader built on top of the Binance `/api/v3/klines` endpoint, which returns standard OHLCV (Open, High, Low, Close, Volume) candlestick data. For each asset, historical data was fetched from its earliest available hourly candle on Binance, resulting in varying start dates across symbols. Bitcoin and Ethereum data is available from August 2017, while more recently listed assets such as Solana and Avalanche begin in mid-2020. All data extends through May 2026. An overview of the dataset, including earliest availability dates and approximate peak market capitalisations, is presented in Table 4.1.

The primary resolution used throughout this research is the one-hour candlestick interval, which offers a balance between granularity and signal-to-noise ratio suitable for the range of AI methods employed. For visualisation and certain indicator computations, hourly data is resampled to daily candles using standard aggregation rules: the opening price of the first hourly candle, the maximum high, the minimum low, the closing price of the final candle, and the sum of volume across all candles within the day. Figure 4.1 presents the full

Table 4.1: Dataset overview: ten cryptocurrency assets selected by historical peak market capitalisation. Peak capitalisation is approximated as the all-time high price multiplied by current circulating supply. Data sourced from Binance REST API and CoinGecko.

Asset	Data From	ATH Price (USD)	ATH Date	Peak Cap (approx.)
BTC	2017-08-17	$126,080.00	2025-10-06	$2,524.6B
ETH	2017-08-17	$4,946.05	2025-08-24	$596.9B
BNB	2017-11-06	$1,369.99	2025-10-13	$184.7B
XRP	2018-05-04	$3.65	2025-07-18	$225.6B
SOL	2020-08-11	$293.31	2025-01-19	$169.0B
ADA	2018-04-17	$3.09	2021-09-02	$114.3B
DOGE	2019-07-05	$0.73	2021-05-08	$112.7B
AVAX	2020-09-22	$144.96	2021-11-21	$62.6B
DOT	2020-08-18	$54.98	2021-11-04	$92.5B
LINK	2019-01-16	$52.70	2021-05-10	$38.3B

Bitcoin price history on a logarithmic scale, illustrating the characteristic bull and bear market cycles, the impact of the third and fourth halving events, and the corresponding evolution of trading volume over time.

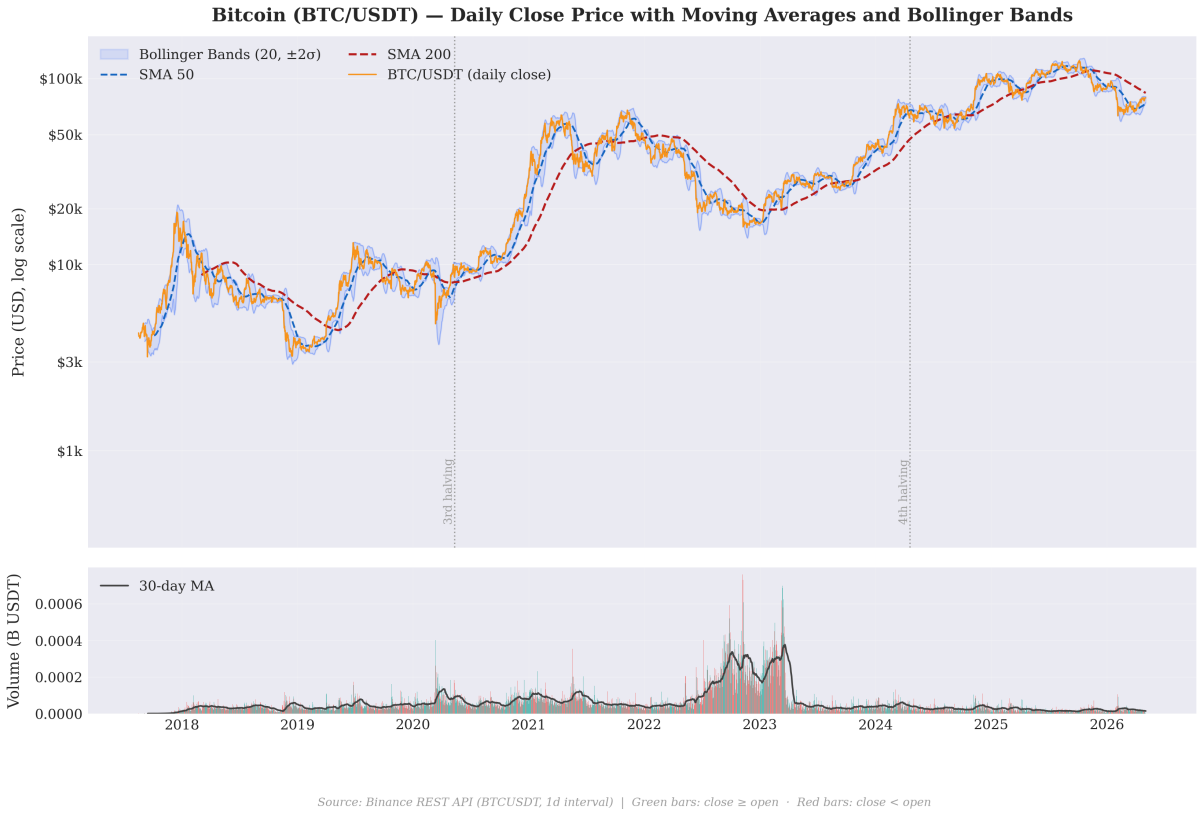

Figure 4.1: Bitcoin (BTC/USDT) daily close price on a logarithmic scale from August 2017 to May 2026, with SMA 50, SMA 200, and Bollinger Bands (20-period, $\pm 2\sigma$). The lower panel shows daily trading volume in billions of USDT, colour-coded by candle direction, with a 30-day moving average overlay. Vertical dotted lines mark the third (May 2020) and fourth (April 2024) Bitcoin halving events.

To ensure data integrity, the downloader implements an incremental update mechanism: upon re-execution, only candles newer than the most recently stored timestamp are fetched from the API, with all data persisted locally in the Apache Parquet format. This avoids redundant network requests and guarantees that the dataset remains consistent across training runs. A single Parquet file per asset stores the complete OHLCV history and serves as the sole input to all downstream preprocessing and feature engineering steps.

4.2 Feature Engineering

4.2.1 Unified Feature Pipeline

All feature data for this research is stored in a single, consistently indexed parquet file produced by a self-contained, phase-by-phase ingestion pipeline. This unified design replaces the earlier practice of loading multiple separate files in each model stage, which introduced version drift and complicated dependency tracking. The pipeline is cache-aware: each phase checks whether its output already exists before computing, allowing re-runs to skip completed phases and enabling partial regeneration when a single feature group is corrected.

A note on the cost and value of breadth is in order here, because feature construction proved to be one of the most demanding parts of the project. The one-hour resolution is a real constraint — it is too coarse to expose order-book microstructure, yet fine enough that a vast number of indicators can be computed from it. This made it easy to *generate* features and hard to generate *useful* ones: the candidate pool grew to several hundred columns, of which only a minority carried any stable predictive content. Moving beyond the handful of features that can be derived directly from raw OHLCV — into multi-timeframe structure, microstructure proxies, cross-asset flow, and sentiment — was the step that made it possible to train anything meaningful at all. On the supervisor's advice, a disciplined, advanced feature-selection stage was therefore treated as essential rather than cosmetic, and is described in Section 4.3.2. The breadth of the problem had a compensating benefit: it forced an exploration of a wide range of state-of-the-art time-series

methods, from gradient boosting to selective state-space models, which is itself one of the contributions of this thesis.

The unified parquet merges five source libraries into a single, consistently indexed hourly time series spanning August 2017 to May 2026:

- **technical library** (196 columns): returns, volatility, moving averages, oscillators, Ichimoku, SuperTrend, Fibonacci, pivot levels, candle patterns, Hurst exponent, halving cycle, and composite regime signals. Computed entirely from raw BTC/USDT OHLCV.
- **external library** (21 columns): fear & greed index (alternative.me), cross-asset ratios (ETH/BTC), altcoin breadth, BTC and ETH market dominance, total crypto market cap change, and stablecoin share from CoinGecko. Computed from external API feeds with a mandatory one-day lag (see Section 4.2.2).
- **microstructure and regime library** (25 columns): taker flow imbalance, true VWAP deviation, average trade size, Hurst exponent at 24h and 72h windows, ADF stationarity statistics, and a binary sideways-flag. Computed from Binance taker-volume feeds.
- **Structural library** (39 columns): swing highs and lows, anchored VWAP at daily and weekly resolution, Point of Control from volume-at-price profiles, Bollinger-Keltner squeeze, and multi-timeframe alignment scores.
- **Microstructure library** (4 columns): Roll spread, Amihud illiquidity, volume imbalance, and sample entropy (SampEn), providing regime-quality signals from price-impact and serial-correlation structure.

After removing OHLCV columns (used by the backtester but not as model inputs), the directional label, and two columns excluded on data-quality grounds (`sma_200` is a raw price level; `mkt_stablecoin_pct` has only 11% non-null coverage due to CoinGecko data availability), the ML-eligible feature pool contains **285 candidate features**.

4.2.2 Leakage Audit

Of all the risks in a project of this kind, the most insidious is a feature that quietly looks into the future. Such a feature does not announce itself: it simply makes the model appear excellent. Midway through development, several configurations produced strikingly promising results that, on inspection, turned out to rest on a small number of contaminated features — daily aggregates that had been forward-filled onto hourly bars without a lag, so that a bar effectively carried information from later in the same day. These false positives were a sobering reminder that in financial machine learning a result that looks too good is usually a leak rather than a discovery. Every feature was therefore audited explicitly before the final results were trusted.

A systematic forward-leakage audit is run as the final phase of the ingestion pipeline. For each candidate feature, the Pearson correlation with the next-bar log-return r_{t+1} is computed over the full history. A feature is flagged if $|\rho(x_t, r_{t+1})| > 0.05$. This threshold is conservative: for a series with $n \approx 74{,}000$ observations, the 99% confidence interval for an uncorrelated pair is $|\rho| < 0.01$, so the threshold provides a substantial buffer above noise.

The features the audit flagged early on were all of one kind: daily external series — market-cap aggregates, dominance, and sentiment — that had been mapped onto the hourly bars by forward-filling without first being lagged, so that an hourly bar borrowed information from later in its own day. The remedy is uniform: every daily series is shifted by one day before it is aligned to the hourly index, and the few features that remained ambiguous were excluded from model training altogether. After this control process every feature in the final pool clears the audit, so the results reported in this thesis rest on a feature panel that has been explicitly validated to be free of forward-looking information.

A shared feature library of **285 ML-candidate features** is derived from the five source libraries described above, organised into 26 thematic groups covering price dynamics, volume, market structure, sentiment, microstructure, and macro cycle patterns. This library constitutes the broadest input space available to any agent in the system; individual

agents then select or reduce this space according to their own learning mechanisms and computational constraints.

The feature groups are summarised in Table 4.2. Key design choices are highlighted below.

Returns and volatility. Raw and logarithmic price returns are computed at lags of 1, 2, 3, 6, 12, 24, 48, 72, and 168 hours, capturing momentum across intraday and weekly horizons. Volatility is measured via rolling standard deviations of log-returns and the Average True Range (ATR) at 14- and 24-period windows.

Moving averages. The close price is expressed relative to six Simple Moving Averages (SMA) and six Exponential Moving Averages (EMA), at periods of 7, 14, 20, 50, 100, and 200 hours. A separate long-cycle group extends this to periods of 336h, 504h, 720h, 2160h, and 4320h, capturing multi-week and multi-month trend alignment relevant to Bitcoin's macro market cycles.

Oscillators. RSI is computed at periods 7, 14, and 21; Stochastic %K at 14 and 21; Williams %R and Chaikin Money Flow (CMF) at their standard settings. MACD histograms are included for both the standard (12, 26, 9) and faster (5, 13, 4) parameter sets.

Calendar and cycle features. Hour-of-day and day-of-week are sine/cosine encoded to preserve periodicity. Bitcoin's ≈4-year halving cycle is encoded as raw position (0–1) and as sine and cosine components, capturing macro seasonality absent from traditional financial data.

Statistical properties. Rolling skewness, kurtosis, and autocorrelation at lags 1h, 6h, 12h, and 24h describe the local return distribution shape. The Hurst exponent over a 168-hour window distinguishes trending ($H > 0.5$) from mean-reverting ($H < 0.5$) regimes.

Additional technical groups cover Bollinger Bands, Ichimoku Cloud, SuperTrend signals at three multiplier values, Fibonacci retracement proximity, daily pivot levels, round-number distances, and composite scores combining momentum, volume, and regime signals. The external, microstructure, structural, and microstructure libraries add further signal diversity as described in Section 4.2.1.

Table 4.2: Feature groups in the unified ML-candidate pool (285 total). All features are computed without look-ahead bias; daily-frequency external features are lagged by one day before being forward-filled to hourly bars.

Group	**N**	**Key indicators**
Technical (196 features from OHLCV)		
Returns	18	Raw and log returns at 1h, 2h, 3h, 6h, 12h, 24h, 48h, 72h, 168h
Volatility	10	Rolling log-return std; ATR(14), ATR(24) raw and percentage
Moving Averages	12	Close vs SMA and EMA at 7, 14, 20, 50, 100, 200 hours
Long-Cycle MA	11	Close vs SMA/EMA at 336h, 504h, 720h, 2160h, 4320h; weekly momentum
Bollinger Bands	4	Band width and position, 20- and 50-period windows
MACD	4	Histograms for (12,26,9) and (5,13,4) parameter sets
Oscillators	6	RSI(7,14,21), Stochastic %K(14,21), Williams %R
Volume	9	Volume z-scores and ratios over 12h, 24h, 72h, 168h; OBV z-score
Volume Profile	10	VWAP(24h, 168h), MFI, CMF, A/D z-scores, volume-weighted RSI
Candle Structure	4	Body fraction, upper/lower wick, bullish flag
Candlestick Patterns	8	Engulfing, doji, hammer, shooting star; bull/bear streaks; marubozu
Price Position	3	Close within 24h, 48h, 168h high–low range
MA Crosses	9	SMA pair distances, golden/death cross flag, recency, ribbon score
Ichimoku	10	TK ratio, cloud position, thickness, cross signals, flip recency

(continued on next page)

Group	N	Key indicators
SuperTrend	8	Direction and distance at mult. 1.5, 2.0, 3.0; consensus; flip recency
Fibonacci	10	Grid position, nearest level distance, 0.618 proximity (48h and 168h)
Support/Resistance	13	Daily pivots P/R1/R2/S1/S2; round-number proximity; breakout magnitude
Volatility Regime	11	Garman-Klass, Parkinson vol; vol-of-vol; ATR percentile; BB squeeze
Divergences	5	RSI, MACD, OBV, and volume-price divergence signals
Statistical	13	Hurst(168h), skewness, kurtosis, autocorrelation (1h–24h)
Calendar	11	Hour and day-of-week sin/cos; halving cycle position, sin, cos
Composite	7	Trend score, RSI×volume confirmation, momentum coherence, Sharpe(24h)
External — daily data, lagged 1 day (20 features)		
Sentiment	3	Fear & Greed index, 7-day MA, 7-day change (alternative.me)
Cross-Asset	6	ETH/BTC ratio, altcoin breadth, BTC relative strength, correlation
Market Structure	4	BTC and ETH dominance, dominance 7-day change, stablecoin fraction
Enhanced OHLCV	4	Vol term structure, normalised momentum (24h and 72h), intraday micro
Microstructure	3	Amihud illiquidity, Kyle lambda, Roll spread (24-bar rolling)
Microstructure and Regime (25 features)		

(continued on next page)

Group	N	Key indicators
Taker Flow	7	TFI, z-scored TFI (24h/72h/168h), avg trade size and deviation
VWAP and Basis	5	True VWAP deviation, taker price premium, buy/sell price spread
Regime Signals	8	Hurst(24h/72h), ADF stat (168h/336h/720h), BB width %, sideways flag
Fracdiff	1	Fractionally-differenced log-price ($d = 0.2$)
Structural (39 features)		
Price Structure	10	Swing highs/lows (minor and major); proximity flags
Liquidity Profile	12	Anchored VWAP (daily, weekly, 24h, 168h); POC distance; volume spikes
Volatility Struct.	8	ATR(20h/72h), Garman-Klass vol, Bollinger-Keltner squeeze
Multi-Timeframe	9	4h and daily EMA spread, RSI, alignment score; session encodings
Microstructure Library (4 features)		
Complexity	4	Roll measure(50), Amihud(50), volume imbalance(50), SampEn(48)
Total ML-eligible	**285**	After excluding OHLCV, label, `sma_200`, `mkt_stablecoin_pct`, `mkt_total_mcap_chg_24h`

Extended feature groups: microstructure and structural features. Beyond the 196 technical indicators, two additional feature libraries enrich the input space for deep learning agents that can leverage higher-resolution signals.

The *microstructure and regime* library (25 columns) is computed from Binance's taker-volume feed and mark/index price klines. It includes: Taker Flow Imbalance (TFI) and its

z-scored rolling variants at 24h, 72h, and 168h — a direct measure of buy- versus sell-side aggressor pressure; average trade size and its normalised deviation from baseline, which is used to identify institutional block activity; the true VWAP derived from quote volume and the close-price deviation from it; the taker price premium (the difference between buy- and sell-side average trade prices, measuring urgency of market participants); Futures mark-price basis and premium, capturing the funding-rate signal; and index-price lead/lag, which measures how far the spot price has deviated from the global spot index. Regime features include the Hurst exponent at 24h and 72h windows, ADF test statistics over 168h, 336h, and 720h windows, the Bollinger Band width percentage, and a binary sideways-flag derived from the combination of low Hurst exponent and narrow volatility. The *structural* library (39 columns) captures market-microstructure geometry. It includes confirmed swing high and low anchors at both minor (12-bar) and major (48-bar) scales, expressed as price distances and binary proximity flags; rolling Point of Control (POC) from volume-at-price profiles; anchored VWAP at daily, weekly, 24h, and 168h resolutions with rolling z-score and exhaustion-spike flags; ATR at 20h and 72h periods; Bollinger–Keltner channel squeeze indicator; Garman–Klass realised volatility; and multi-timeframe (4h and daily) EMA spread, RSI, and composite alignment score.

Agent-specific feature selection pipelines. A key architectural decision in the system is that each agent operates on a *different subset* of the shared feature library, selected according to the agent's learning mechanism, capacity, and role in the ensemble. This deliberate heterogeneity serves two purposes: it avoids redundancy between agents that would reduce the diversity of the ensemble, and it allows each agent to use only the features that are well-matched to its inductive bias. Table 4.3 summarises the per-agent feature diets.

The LightGBM agent's selection is data-driven end-to-end: the relevance–redundancy filter runs on all pre-OOS data and produces a working set from the candidate pool (Section 4.3.2). The Mamba diet is also data-selected, by a Boruta-against-TBM procedure on a narrower technical and microstructure pool tailored to its state-space objective. The TCN and PatchTST agents share an identical 32-feature curated diet, which allows a direct controlled comparison of their architectures on the same input representation.

Table 4.3: Feature set used by each of the four learned agents. The selections differ in motivation: LGBM applies a relevance–redundancy filter to the full candidate pool; Mamba uses a Boruta-against-TBM sequential diet; and the TCN and PatchTST share an identical curated 32-feature sequential diet. The three rule-based agents are not included here, since they read a small, fixed set of features rather than a learned selection.

Agent	**Features**	**Selection approach and composition**
LightGBM	20–50	Relevance–redundancy filter from all 285 ML-eligible features (pre-OOS only): coverage/variance screen, direction-agnostic univariate AUC filter (> 0.502, 199 survivors), greedy Spearman redundancy pruning at $\rho_{\max}$, truncation to top N_{feat}. $N_{\text{feat}} \in \{20, 35, 50\}$ and $\rho_{\max} \in \{0.85, 0.90\}$ are tuned in the joint grid (Section 4.3.3). Leaking feature `mkt_total_mcap_chg_24h` excluded.
Mamba SSM	39	Boruta-against-TBM selection on technical and microstructure features: 39 confirmed features including long-horizon returns (48h–168h), Fibonacci proximity, MA ribbon, Ichimoku cloud, Hurst, kurtosis, variance ratio, ADF stationarity and Bollinger width.
TCN	32	Curated sequential diet: 9 LGBM-core features + 11 technical regime/volatility features + 7 microstructure (TFI, Hurst, VWAP deviation) + 4 structural (VWAP dev, ATR, multi-timeframe alignment) + fractionally-differenced log-price. Sequence length: 48 bars.
PatchTST	32	Identical 32-feature diet to TCN (same curated selection of technical, microstructure, structural, and fracdiff features). Sequence length: 48 bars; patched into 11 overlapping segments of 8 bars with stride 4.

This design lets the system be extended with new feature groups without retraining existing agents: a new library can be introduced into the LGBM filter or added to the Mamba, TCN, or PatchTST diet while leaving the other agents unchanged.

4.3 LightGBM Agent

4.3.1 Method

LightGBM [11] is a gradient boosting framework that builds an ensemble of decision trees using a leaf-wise growth strategy with histogram-based binning. Compared to level-wise boosting, leaf-wise growth tends to produce deeper, more expressive trees with fewer it-

erations, and scales efficiently to large datasets and high feature dimensionality. These properties make it well suited to the tabular, heterogeneous feature set described in Section 4.2.

The prediction task is formulated as binary classification: at each hourly candle, the model estimates the probability $\hat{p} = P(\text{close}_{t+1} > \text{close}_t)$. A value $\hat{p} \geqslant \tau_{\text{long}}$ triggers a long entry; $\hat{p} \leqslant \tau_{\text{short}}$ triggers a short entry. The model output is thus a continuous confidence score rather than a hard label, which enables threshold-based signal filtering.

4.3.2 Feature Selection Pipeline

Reducing the 285-candidate pool to a compact working set is essential rather than cosmetic. High-dimensional inputs inflate variance and invite spurious in-sample fits — a concern that is acute in financial machine learning, where the signal-to-noise ratio is low and the effective number of independent observations is far smaller than the bar count [9]. Feature selection methods are conventionally grouped into *filter*, *wrapper*, and *embedded* families [16]: filters score features by an intrinsic statistical criterion independent of the learner; wrappers search feature subsets by repeatedly training the model; embedded methods (such as the impurity-based importances of gradient boosting itself) select during training. Wrapper searches over a 285-feature pool are combinatorially infeasible when each evaluation is a full walk-forward run, so we adopt a *filter* pipeline whose two free parameters — the retained set size and the redundancy threshold — are then tuned jointly with the model hyperparameters (Section 4.3.3). The pipeline is fitted strictly on pre-OOS data, so no information from the test window enters selection.

The filter is a *relevance–redundancy* design, the same principle that underlies correlation-based feature selection [17] and the minimum-redundancy–maximum-relevance (mRMR) criterion [18]: a good subset contains features that are individually predictive of the label (relevance) yet mutually decorrelated (low redundancy). It proceeds in four stages.

1. **Coverage and variance screen.** Features with fewer than 1,000 non-null pre-OOS observations or with near-zero variance are discarded, as they cannot contribute a stable split.

2. **Univariate relevance via direction-agnostic AUC.** Each feature's ROC-AUC against the binary directional label is computed on the pre-OOS window. Because a feature may be informative in either direction, the *effective* relevance is taken as $\max(\text{AUC}, 1-\text{AUC})$, exploiting the symmetry of the AUC under label inversion [19]. Features with effective AUC $\leqslant 0.502$ are rejected. For a series with $n \approx 74{,}000$ near-balanced labels, the standard error of an uninformative AUC is $\approx 1/\sqrt{2n} \approx 0.003$, so the 0.502 cut sits just below one standard error above the 0.5 noise floor — deliberately permissive, since the redundancy and truncation stages perform the stronger pruning. This stage retains 199 of the 279 features that clear the coverage screen.

3. **Greedy redundancy pruning.** The surviving features are sorted by descending effective AUC and admitted one at a time; a candidate is rejected if its absolute Spearman rank correlation with any already-admitted feature exceeds the threshold $\rho_{\max}$. Sorting by relevance before pruning guarantees that, between two collinear features (e.g. SMA and EMA at the same period, or RSI at adjacent look-backs), the *more* predictive one is the survivor — the explicit relevance–redundancy trade-off of CFS/mRMR realised as a single greedy pass. Spearman rather than Pearson correlation is used so that monotone non-linear redundancies are also caught.

4. **Relevance truncation.** The decorrelated, relevance-ranked list is truncated to its top N_{feat} entries, yielding the final working set passed to the walk-forward training loop.

The two filter parameters are not fixed *a priori*. The retained-set size $N_{\text{feat}} \in \{20, 35, 50\}$ and the redundancy threshold $\rho_{\max} \in \{0.85, 0.90\}$ are exposed to the joint grid search of Section 4.3.3, so the selection aggressiveness is itself optimised against downstream trading performance rather than guessed. This couples the filter to the embedded importances of the gradient-boosted model that consumes its output, recovering part of the benefit of a wrapper search at a fraction of its cost. A more exhaustive permutation-based confirmation such as Boruta [20] — which contrasts each feature's importance against column-permuted "shadow" copies — was considered but not adopted here: its tens of model fits per candidate are prohibitive inside a grid that already trains 48 walk-forward

configurations. The filter approach is also competitive with more elaborate selectors on high-dimensional classification benchmarks [21], which justifies the trade-off.

4.3.3 Training and Walk-Forward Protocol

The LightGBM agent uses a *sliding* two-year walk-forward scheme (denoted M2Y): each fold trains on the trailing 17,520 hours (two years) and steps forward one month (720 hours), so the training distribution always reflects recent market character rather than accumulating stale regimes from the distant past. Training data preceding each OOS fold is split into a training portion (80%) and a held-out validation tail (20%), the latter used solely for early stopping (patience 50 rounds). All features are filled with zero for missing values before training; no normalisation is applied since gradient boosting is invariant to monotone feature transformations.

A 12-bar embargo is applied between each training window's end and the OOS inference window's start, preventing any information overlap from the boundary between training and prediction.

A joint grid search is conducted over both model hyperparameters and trading execution parameters. The model grid covers five axes ($3 \times 2 \times 2 \times 2 \times 2 = 48$ configurations):

- $N_{\text{feat}} \in \{20, 35, 50\}$ — retained features after selection
- $\rho_{\max} \in \{0.85, 0.90\}$ — correlation filter threshold
- `num_leaves` $\in \{31, 63\}$ — tree complexity
- `min_child_samples` $\in \{30, 50\}$ — leaf regularisation
- `learning_rate` $\in \{0.01, 0.02\}$

For each trained model, the walk-forward probabilities on the test set are computed once and fixed. A second grid over nine trading parameters (asymmetric long/short entry thresholds, limit-entry ATR offset, stop-loss and take-profit ATR multipliers, minimum stop floor, minimum and maximum hold, cooldown) enumerates 5,184 configurations per model, so the joint search evaluates on the order of a quarter of a million backtested strategy variants in total ($48 \times 5{,}184$). This two-level structure avoids retraining the

model for every trading configuration and makes the search computationally feasible. The optimisation target is the annualised Sharpe ratio on the test period, subject to a minimum-trade floor.

A configuration is only admitted if it executes at least **120 trades** over the two-year OOS window (≈ 60 per year). This floor is a deliberate correction to an earlier specification that required only 30 trades: with so weak a floor, the Sharpe-maximising configuration was an ultra-selective one (a high long threshold of 0.63, satisfied by under 2% of bars) that fired only a handful of times and left the equity curve flat for months at a stretch. Although the predicted probabilities cross actionable thresholds throughout the test window — over a thousand bars exceed 0.58, distributed across all three regimes — the unconstrained search nonetheless preferred a near-dormant policy whose high Sharpe rested on a tiny, fragile trade sample. Requiring a minimum trading frequency forces the search toward configurations that remain continuously engaged with the market, which both stabilises the Sharpe estimate and produces a return stream of practical use to the downstream fund allocator.

The distribution of performance across these configurations, and the gap between the typical model and the best one, is examined in Chapter 6. A practical advantage of gradient boosting is worth noting here: once each model is trained, computing its predictions over the whole test set and scoring thousands of trading configurations is almost instantaneous, so the entire search completes in minutes on a standard laptop.

4.4 TCN Agent

4.4.1 Method

The TCN agent is a deep learning model based on the Temporal Convolutional Network (TCN) architecture [12], which processes multi-feature time-series windows using stacked dilated causal convolutions. Unlike LightGBM, which operates on each candle independently as a tabular feature vector, the TCN receives the full historical context as a sequence, allowing it to learn temporal patterns such as momentum build-up, volatility clustering, and multi-bar candlestick formations directly from the raw feature time series.

The model follows a multi-task formulation: the primary task is three-class directional classification (up/flat/down) and the auxiliary task is forward realised volatility regression. The auxiliary task provides a richer gradient signal during early training, when the classification objective is dominated by the random-baseline cross-entropy, and acts as an implicit regulariser that prevents the shared encoder from overfitting to directional noise.

4.4.2 Labelling: Triple Barrier Method

Rather than using simple next-bar returns as the prediction target, the TCN agent uses the Triple Barrier Method (TBM) [9]. For each bar t, three barriers are placed:

- **Upper barrier** at $+k\sigma_t$ above the close, where σ_t is a 24-bar rolling standard deviation of log-returns and $k = 2.0$;
- **Lower barrier** at $-k\sigma_t$ below the close (symmetric);
- **Vertical barrier** after $T_{\max} = 24$ hours.

The label is determined by whichever barrier is touched first over the following $T_{\max}$ bars: label 1 (up) if the upper barrier is reached, label 0 (down) if the lower barrier is reached, label 2 (flat) if the vertical barrier expires first. Using $k = 2.0$ volatility multiples produces wider barriers than the raw next-bar return, filtering out microstructure noise and ensuring that labelled directional moves represent economically meaningful events. The resulting label distribution over the training set is approximately 30% down, 31% up, and 39% flat, reflecting the tendency of price to consolidate within barriers most of the time.

Sample weights are assigned inversely proportional to the class frequency and further scaled by a volatility overlay: bars during high-volatility regimes receive larger gradients, directing the model's attention toward the most informative periods.

4.4.3 Feature Normalisation and Sequence Construction

Each feature in the 32-feature input set is normalised independently using a `QuantileTransformer` fitted on the training partition only (no look-ahead bias), mapping the marginal distribution of each feature to a standard normal. This removes

the need for the network to learn feature-specific scales and makes the representation insensitive to outliers, which matters for features such as volume z-scores that can spike by orders of magnitude during market events.

Sequences are constructed by sliding a window of length $T = 48$ bars over the normalised feature matrix. Each sequence therefore covers 48 hours (two calendar days) of history. The window stride is one bar, so the training set contains one sequence per hourly candle after a 48-bar burn-in. The fractionally-differenced log-price is included as the 32nd feature channel, providing the network with price-level context that plain returns discard while avoiding unit-root non-stationarity.

4.4.4 Architecture

The TCN encoder consists of four sequential blocks, each comprising two dilated causal convolutions with weight normalisation [14] and dropout (rate 0.20), followed by a residual connection. The convolution parameters are: kernel size $k = 3$, output channels $C = 64$, and dilations $\{1, 2, 4, 8\}$ for blocks 1 through 4 respectively. With these parameters the receptive field spans $1 + 2(3 - 1)(1 + 2 + 4 + 8) = 61$ timesteps, comfortably covering the 48-bar input window. The total trainable parameter count is approximately 96,000, making the model fast to train and evaluate.

Two task-specific heads are attached to the final hidden state (last temporal position of the encoder output):

- **Classification head**: a two-layer MLP with ReLU activation producing three logits (down / up / flat).
- **Volatility head**: a single linear layer producing a scalar forecast of the AUX_FWD_H = 6-bar forward realised volatility.

The combined loss is:

$$\mathcal{L} = \mathcal{L}_{\text{CE}}^{\text{class-weighted}} \cdot w_{\text{vol}} + \lambda_{\text{vol}} \cdot \mathcal{L}_{\text{Huber}}(\hat{\sigma}, \sigma), \tag{4.1}$$

where w_{vol} is the per-sample volatility weight, $\lambda_{\text{vol}} = 0.5$ controls the auxiliary task strength, and Huber loss is used for the regression task to limit the influence of extreme volatility spikes.

4.4.5 Training

The model is trained using the AdamW optimiser [22] with learning rate 3×10^{-4} and weight decay 10^{-4}, for a maximum of 100 epochs with early stopping (patience 20). A linear learning-rate warmup over the first 5 epochs stabilises early training. The batch size is 256. All training is performed on a single GPU or MPS device where available, with CPU fallback; the relatively compact model size makes the full training run complete in approximately 10–15 minutes.

The data split is aligned with the unified OOS window adopted across all agents: training up to 31 December 2022 (providing 5 years of history), grid-search validation from 1 January 2023 to 30 May 2024 (covering bear and bull sub-regimes), and test from 31 May 2024 onward (the 24-month unified evaluation window). The validation set is not used for gradient updates; it is used exclusively for early stopping and, subsequently, for grid search over trading execution parameters.

4.4.6 Trading Execution and Grid Search

Raw model outputs are three-class probabilities: p_{up}, p_{down}, p_{flat}. These are converted to trading signals using threshold rules: a long entry is triggered when $p_{\text{up}} \geqslant \tau_{\text{long}}$; a short entry when $p_{\text{down}} \geqslant \tau_{\text{short}}$ and $p_{\text{up}} < \tau_{\text{long}}$. Entry, stop-loss, and take-profit levels are set as ATR multiples of the current bar's close price. An exhaustive grid search over all parameter combinations is performed on the validation set; the configuration maximising the annualised Sharpe ratio is applied to the held-out test set. The grid covers 3,456 combinations of nine execution parameters (entry thresholds, ATR multipliers for entry offset, stop-loss, and take-profit, minimum and maximum hold durations, and cooldown period).

The fee model applied to TCN backtests uses limit-entry orders for both long and short trades: an entry limit is placed entry_atr_mult $\times$ ATR away from the current close in the

direction of the trade. If the limit level is touched within the next candle (determined by whether the candle high or low reaches the target, adjusted for a 5 bp buffer), the trade fills at the maker rate (0%); otherwise it is skipped rather than forced to market. Take-profit exits also use limit orders (0% maker). Stop-loss and timeout exits are taker (0.05%). Short positions additionally receive a funding rate credit of +0.00077% per hour.

4.5 Mamba SSM Agent

4.5.1 Method

The Mamba agent is based on the selective state-space model (SSM) architecture [23], which processes sequential data through a learned input-dependent selection mechanism. Unlike the fixed dilated convolutions of the TCN, the Mamba block adapts its state-transition matrices to the current input at each step, selectively retaining or forgetting historical context. This property is theoretically advantageous for financial time series characterised by intermittent regime shifts: the model can amplify the signal from regime-change candles while suppressing noise during consolidation periods.

The agent is implemented in pure PyTorch using a chunked vectorised SSM scan for efficiency, avoiding the requirement for the `mamba-ssm` CUDA kernel library. The chunked scan divides each sequence of length L into $L/\text{chunk\_size}$ groups, computing the SSM recurrence within each group using parallel prefix operations and carrying state across group boundaries sequentially. This provides GPU efficiency comparable to the fused kernel while running on any hardware (CPU, MPS, CUDA).

4.5.2 Architecture

The model consists of an input projection layer followed by $N_{\text{layers}} = 2$ stacked `MambaBlock` modules and a binary classification head. Each block contains:

- An input projection to the inner dimension $d_{\text{inner}} = d_{\text{model}} \times \text{expand} = 64 \times 2 = 128$.
- A depthwise 1D causal convolution with kernel size $d_{\text{conv}} = 4$, providing local context.

- The selective SSM scan with state dimension $d_{\text{state}} = 16$, producing the sequence output.
- A residual connection around the entire block.

The total parameter count is approximately 160,000, and a single forward pass on a $(\text{batch}, 24, 39)$ input tensor completes in approximately 4 ms per batch on an A100 GPU.

4.5.3 Feature Input and Labelling

The Mamba agent operates on 39 features selected by the 4-stage Boruta pipeline (see Section 4.3) applied against the Triple Barrier Method label (TP=1, SL=0 with $\pm 1.5\times$ATR barriers and a 24-bar time limit). This Boruta-against-TBM procedure naturally selects features predictive of the actual trade-level outcome, aligning the feature selection target with the model's trading objective. The input sequences have length SEQ_LEN = 24 bars. All features are normalised using a `QuantileTransformer` fitted on each fold's training window only, mapping marginal distributions to standard normal. This removes the need to engineer scale-invariant representations and provides robustness to extreme values.

4.5.4 Training and Walk-Forward Protocol

Unlike the TCN (single train pass) and the LightGBM agent (a sliding two-year window), the Mamba agent uses an *expanding-window* walk-forward: the training set always begins at the start of the available history (January 2019) and grows with each fold, while the model is retrained every 3 months. This lets the state-space model exploit the full history — its selection mechanism is meant to suppress stale-regime data internally — while still being re-fitted frequently enough to track the changing market character.

Training uses `AdamW` with $\text{lr} = 3 \times 10^{-4}$, a linear warmup over 5 epochs followed by cosine annealing to 10^{-6}, and FocalLoss with $\gamma = 2.0$ to down-weight easy (near-50%) probability examples. Early stopping with patience 8 is applied on the fold's validation AUC. The model is trained on Google Colab with an A100 GPU; the full multi-fold walk-forward requires 30–45 minutes.

After the WFO loop, fold-specific predictions are stitched to produce a continuous $P(\text{up})$ time series aligned to the hourly index. The same ATR-bracket backtester and grid-search

pipeline used for LGBM and TCN is applied identically, enabling direct performance comparison across the three agents.

4.6 PatchTST Agent

4.6.1 Method

The PatchTST agent is based on the patch-based Transformer encoder introduced by Nie et al. [24] for time-series forecasting, adapted here for binary directional classification. The core motivation for this architecture is to overcome a limitation shared by both the TCN and the Mamba agent: their respective receptive fields are determined by architectural hyperparameters (dilation schedule and sequence length), and they process features at individual bar granularity. PatchTST instead partitions the input sequence into coarser segments (patches), allowing the self-attention mechanism to relate non-adjacent market segments without paying the quadratic attention cost over all individual bars.

Unlike the original forecasting formulation, the PatchTST agent follows the same classification objective used by the TCN and Mamba agents: given a 48-bar feature window, estimate $\hat{p} = P(\text{close}_{t+1} > \text{close}_t \mid \text{TBM\_TP})$, where the label is generated by the Triple Barrier Method. The use of an identical input representation to the TCN (same 32-feature diet, same sequence length) ensures that performance differences can be attributed to the architectural choice rather than the input.

4.6.2 Patch Tokenisation

The input tensor $\mathbf{X} \in \mathbb{R}^{L \times F}$ (sequence length $L = 48$, features $F = 32$) is divided into overlapping patches by a sliding window of length $P = 8$ with stride $S = 4$, producing $N_p = \lfloor (L-P)/S \rfloor + 1 = 11$ patches per feature channel. Each patch $\mathbf{x}_i \in \mathbb{R}^{P \times F}$ is flattened and linearly projected to a $d_{\text{model}} = 64$ dimensional embedding:

$$\mathbf{e}_i = \mathbf{W}_{\text{proj}} \, \text{vec}(\mathbf{x}_i) + \mathbf{b}_{\text{proj}}, \quad \mathbf{W}_{\text{proj}} \in \mathbb{R}^{d_{\text{model}} \times PF}. \tag{4.2}$$

A learnable positional embedding $\mathbf{p}_i \in \mathbb{R}^{d_{\text{model}}}$ is added to $\mathbf{e}_i$ to encode the patch's temporal position within the window. This patching scheme reduces the sequence length seen by

the Transformer from 48 bars to 11 patch tokens, keeping the self-attention computation tractable while preserving the temporal structure at an 8-bar granularity.

4.6.3 Architecture

The Transformer encoder consists of 2 stacked layers, each with multi-head self-attention ($h = 4$ heads, head dimension $d_k = 16$) and a position-wise feed-forward sub-network with inner dimension $d_{\text{ff}} = 128$. Layer normalisation is applied before each sub-layer (pre-norm formulation) and dropout with rate 0.2 is applied after each sub-layer and in the attention weights. The output of the final encoder layer is mean-pooled over the 11 patch tokens to produce a single $d_{\text{model}} = 64$ dimensional representation, which is passed through a linear classification head to produce a raw logit.

Temperature scaling. After training, the logit is divided by a learned scalar temperature $T = 1.625$ before applying the sigmoid function. The temperature is calibrated post-hoc on the validation set by minimising the expected calibration error (ECE), following Guo et al. [25]. The calibrated probability is:

$$\hat{p} = \sigma\left(\frac{z}{T}\right), \quad T = 1.625. \tag{4.3}$$

Temperature scaling does not alter the ranking of predictions and therefore has no effect on AUC; it solely improves the reliability of $\hat{p}$ as an absolute probability estimate, which matters when the agent's confidence drives its risk-managed position size.

The total parameter count is 84,547, which is smaller than the TCN (≈96,000 parameters) and reflects the shared weights in the self-attention layers relative to the per-dilation convolutional filters of the TCN.

4.6.4 Labelling

PatchTST uses the same Triple Barrier Method labelling as the TCN and Mamba agents, with barriers placed at $\pm 2.0 \times \sigma_{24}$ (where σ_{24} is the 24-bar rolling standard deviation of log-returns) and a vertical barrier at 24 bars. The TBM generates a binary label: 1 if the upper barrier is touched first (profitable direction), 0 otherwise.

4.6.5 Feature Normalisation

Features are normalised with a `QuantileTransformer` (mapping each feature's marginal distribution to a standard normal) fitted on the training window only. This is the same scaling used by the TCN and Mamba agents, and it is well suited to the several heavy-tailed PatchTST inputs (e.g., `hurst_168h`, `bb_width_pct`), whose extreme values are compressed by the rank-based mapping.

4.6.6 Training Protocol

The PatchTST agent uses a *single chronological split* rather than walk-forward cross-validation: training data covers August 2017 to December 2022, the validation set covers January 2023 to May 2024 (for hyperparameter optimisation and early stopping), and the held-out test set covers June 2024 to May 2026. This matches the LGBM and TCN evaluation protocol and allows a unified OOS period across all three agents that use static train-test splits.

Training uses `AdamW` with lr $= 5 \times 10^{-4}$, weight decay 10^{-4}, and a cosine annealing schedule over 80 epochs with linear warmup for the first 5 epochs. Like the TCN, the model is trained multi-task — a directional head and an auxiliary forward-volatility head — with binary cross-entropy on the direction target. Early stopping with patience 20 monitors validation loss; the best model (epoch 12, validation loss = 0.693) is retained. The OOS AUC on the test set is 0.502.

4.6.7 Trading Execution and Grid Search

The PatchTST trading pipeline is identical to that of the TCN: the calibrated probability series $\hat{p}$ is evaluated against a grid of long and short entry thresholds, ATR-based stop-loss and take-profit multipliers, minimum and maximum hold durations, and a cooldown period. The optimal configuration identified on the validation set is: $\tau_{\text{long}} = 0.58$, $\tau_{\text{short}} = 0.58$ (symmetric), entry_atr_mult = 0.6 (limit-order entry), sl_atr_mult = 1.5, tp_atr_mult = 3.0, min_hold = 12, max_hold = 24, cooldown = 2 bars.

The symmetric thresholds at 0.58 impose a conservative activation criterion: only signals with $\hat{p} \geqslant 0.58$ (long) or $\hat{p} \leqslant 0.42$ (short) are acted upon. Given the calibrated nature

of the PatchTST output (temperature-scaled), these thresholds correspond to genuine probability bounds rather than raw logit cutoffs.

4.7 Rule-Based Agents

The four learned agents are all non-linear models fitted to overlapping views of the same feature panel, so they share failure modes: when the market moves in a way none of them learned, they tend to be wrong together. To add genuinely orthogonal behaviour, three classical *rule-based* agents are introduced, whose edge — where it exists — comes from economic logic rather than from fitting the data. Each computes a causal, parameter-free conviction in $[0, 1]$ (0.5 neutral) as a vote fraction over already-causal features using fixed thresholds, so no statistic is fitted on the data and there is no scaler or label that could leak future information.

Trend votes on the bullishness of the moving-average, SuperTrend, and MACD structure: a long when these filters agree upward, a short when they agree downward, neutral otherwise. **Volatility breakout** trades the direction of a confirmed 24-hour range break, with full conviction when the break follows a recent volatility squeeze and half conviction otherwise. **Dominance rotation** reads cross-asset capital flow: it leans long Bitcoin when the seven-day change in BTC dominance is positive while the ETH/BTC cross weakens (capital rotating into BTC), and short or neutral in the opposite, alt-season configuration. The daily dominance series is lagged by 24 hours before being used on hourly bars, precisely to avoid the same-day market-cap leakage discussed in Section 4.2.2.

Two further rule agents — a contrarian Fear & Greed sentiment agent and a mean-reversion agent — were implemented but *excluded* from the final set of agents because they produced negative out-of-sample returns; the sentiment agent additionally fell below its random-bracket null (Section 4.8). They are reported as honest negative results in Chapter 6.

Crucially, the rule agents are not fitted: only their ATR-bracket execution parameters (entry, stop-loss and take-profit multiples, hold limits, thresholds) are grid-searched on the 2022-01–2024-05 validation window by validation Sharpe, and frozen before the hold-out is touched — the identical protocol used for the learned agents. The same `bracket_run`

execution engine is used verbatim, so each agent reproduces bit-for-bit when the fund rebuilds it.

4.8 Capital Allocation and Coordination

The integration layer treats each of the seven agents as an autonomous sub-strategy delivering an independent stream of realised returns, and its task is to allocate capital across those streams without look-ahead.

Final allocator: capped inverse-volatility risk parity. At each rebalancing step the weight of agent k is set inversely proportional to the trailing volatility of its return stream, $w_k \propto 1/\hat{\sigma}_k$, estimated strictly on data preceding the step. Each weight is then capped at $2/N$ — with $N = 7$ agents, a cap of $\approx 28.6\%$ — and the weights are renormalised. Inverse-volatility weighting equalises each agent's *risk* contribution rather than its capital, giving quieter agents more weight and turbulent ones less; the cap prevents any single method from dominating the fund through a stretch in which only it happens to be active. The rule is fully causal and interpretable, and it deliberately sacrifices part of the uncapped inverse-volatility return in exchange for a more credible, diversified allocation. The naive equal-weight fund ($w_k = 1/N$) is reported alongside it as a baseline.

Ablation: a regime-gated adaptive coordinator. For comparison we also implement the kind of adaptive controller one might expect to be the centrepiece of an "intelligent" multi-agent system: weights proportional to a softmax of each agent's trailing Sharpe ratio multiplied by a per-regime competence score estimated on pre-OOS data, where regimes (chop / bull / bear) are labelled from stationary trend and volatility features only. The same detector is applied identically before and during the hold-out, so the competence priors are leak-free. As Chapter 6 reports, this coordinator did *not* beat the simple capped inverse-volatility rule; it is retained as an instructive negative ablation rather than as the headline mechanism.

Random-bracket null test. Because an asymmetric ATR bracket (a take-profit wider than the stop) can manufacture large returns from an essentially random signal, a strong

return alone does not establish forecasting skill. Each agent's signal is therefore compared against a *random-bracket null*: its realised return is re-evaluated under many random permutations of the signal (preserving its value distribution) fed through the same bracket. An agent whose true return sits inside the bulk of this null distribution has no demonstrated timing edge even if its headline return is high. This test is the basis on which the dominance-rotation agent — which posts a large return but sits at roughly the 95th percentile of its own null — is described as a diversification agent rather than an alpha source, and on which the cross-asset and sentiment experiments were rejected.

5 Implementation

5.1 Software Stack

The system is implemented entirely in Python, organised as a single repository in which a shared core package provides the common utilities — data loading, feature engineering, agent interfaces, and the backtesting engine — and a sequence of self-contained notebooks drives the experiments. Each notebook is generated automatically from the corresponding source modules, so the university-facing notebooks and the maintained runtime code cannot drift apart. The repository is available privately and is described in Section 5.2.

Key libraries.

- **Data:** `pandas` and `numpy` for processing, `pyarrow` for columnar (Parquet) storage, and `requests` for the Binance and alternative.me APIs.
- **Machine learning:** `lightgbm` for the gradient-boosting agent, `scikit-learn` for preprocessing, metrics, and feature selection, and `scipy` for the statistical tests.
- **Deep learning:** `PyTorch` for the TCN, Mamba, and PatchTST models, their custom losses, and data loaders. The Mamba selective-scan is implemented in pure PyTorch without external CUDA kernels, so it runs unchanged on CPU, Apple MPS, or NVIDIA GPUs.
- **Feature engineering:** `statsmodels` for the Augmented Dickey–Fuller test, `numba` for the accelerated Hurst-exponent computation, and custom implementations of all technical indicators.

5.2 Reproducible Pipeline

A central engineering goal of the project is that every reported number can be regenerated end-to-end from raw data. The pipeline is a linear, cache-aware sequence of stages: ingestion, the four learned agents, the rule-based agents, and the final fund. Each stage reads the unified feature file, performs its walk-forward training and grid search, and writes a standardised set of artifacts; the final stage consumes those artifacts, wraps each as an autonomous agent, compares the allocation rules, and emits the thesis figures. Because every stage is cached by output existence, a corrected feature group or a re-tuned agent triggers only the minimal recomputation downstream.

Standardised artifact contract. Every producing stage writes the same schema: a `float32` array of out-of-sample probabilities, the matching integer-nanosecond timestamp index, the equivalent full-history (walk-forward) arrays, a `results.json` performance record, and the trained model. Fixing this contract is what allows the agents to be developed independently yet combined mechanically by the final stage. The timestamp encoding is made explicit as integer nanoseconds to avoid a unit-ambiguity defect that arises when a microsecond- or millisecond-resolution index is cast to integer in recent `pandas` versions — a subtle bug that, left unguarded, would misalign an agent's signal against the price series by three orders of magnitude.

Shared backtesting engine. All agents share a single ATR-bracket backtester: a one-pass loop over the OOS index tracking the open position, any pending limit order, the stop-loss and take-profit levels, the holding count, a cooldown counter, and accrued short funding. A signal at bar t only places a *pending* order that can fill at bar $t + 1$, so no signal ever acts on the same bar's price; exits are checked only after a minimum hold has elapsed. Using identical execution code across every agent guarantees that performance differences reflect signal quality rather than implementation artefacts.

5.3 Computational Requirements and Hardware

Most of the work runs comfortably on a personal machine. The gradient-boosting agent's walk-forward generates on the order of a hundred small model fits, each a few seconds, and its entire model-plus-trading grid search completes in minutes; the rule agents and the fund layer are similarly light. All of this, together with the TCN, was developed and trained locally on a MacBook Pro (M4 Pro, 2024, 48 GB unified memory) using the Apple MPS backend, on which a full TCN training run takes roughly fifteen to twenty-five minutes.

The selective state-space (Mamba) agent was the exception. Its expanding walk-forward retrains the model many times over, and despite the capable laptop this was impractical to iterate on locally; it was therefore trained on Google Colab using a single A100 GPU, under a paid subscription of the same kind used for the author's earlier (bachelor's) thesis. A complete walk-forward there takes roughly thirty to forty-five minutes. It was a useful reminder that even a fast laptop sometimes cannot replace a proper GPU, and that the hardware on hand quietly decided which experiments were realistic to run.

6 Experiments and Results

6.1 Evaluation Framework

All agents are evaluated on a unified two-year held-out test period, **31 May 2024 – 31 May 2026**. This window was chosen because it spans three structurally distinct market regimes that together provide a comprehensive stress test of any active strategy:

Table 6.1: Three market regimes within the two-year OOS evaluation period. Each regime presents a distinct challenge for active trading strategies.

Regime	**Dates**	**Market Condition**	**Strategy Challenge**
Chop	31 May – 5 Nov 2024	Sideways, high noise; pre-election uncertainty with violent range-bound oscillations and no net direction.	Avoid over-trading and fee drag during choppy, low-signal conditions.
Bull	6 Nov 2024 – 31 Oct 2025	Parabolic bull market; post-election price discovery; BTC reached an all-time high above $126,000.	Capture trend momentum; avoid premature stop-outs; compare with buy-and-hold.
Bear	1 Nov 2025 – 31 May 2026	Macro capitulation; risk-off unwinding dropped BTC over 50% from its all-time high.	Profit or preserve capital through short exposure.

The choice of this particular window is itself deliberate, on three counts. It is long enough — almost two years of hourly data — to give a statistically meaningful number of trades rather than a handful of lucky ones. It contains three genuinely different market conditions, so a strategy must prove itself in more than one environment. And, just as importantly, it is *not* one of the early Bitcoin periods in which simply buying and holding the coin would have returned several hundred percent: over those years Bitcoin was an exceptional asset whose appreciation would flatter almost any long-biased strategy and make for a dishonest baseline. Evaluating on a recent, mixed window keeps the buy-and-hold benchmark fair.

The multi-regime structure also tests whether the system's edge is genuine *alpha* — the ability to navigate all three conditions — or merely the result of riding a directional trend during the bull sub-period. A strategy that beats buy-and-hold in the bull but also stays positive through the chop and the bear has shown real market-condition robustness. Performance is reported both for the full window and per regime, using total return, annualised Sharpe and Sortino ratios, maximum drawdown, and alpha over the corresponding buy-and-hold segment (the financial metrics are defined in Chapter 7).

6.2 Individual Agent Performance

Each agent is first evaluated in isolation on the unified window, under *leak-free* selection: its execution parameters are chosen on the pre-OOS validation window and frozen before the hold-out is touched. Table 6.2 reports the canonical figures, all computed on the identical strict window so that the rows are directly comparable. Two passive benchmarks are included: Bitcoin buy-and-hold (the natural crypto benchmark) and the S&P 500 (a broad-market reference).

Table 6.2: Standalone agent performance on the unified OOS window (31 May 2024 – 31 May 2026), net of fees, under leak-free parameter selection. AUC is the out-of-sample directional/Triple-Barrier AUC where applicable. Returns are sorted by annualised Sharpe.

Agent	**AUC**	**Return**	**Sharpe**	**Sortino**	**MaxDD**
PatchTST (patch transformer)	0.502	+74.9%	1.59	0.94	−17.3%
LightGBM (gradient boosting)	0.544	+89.7%	1.50	0.83	−12.9%
Dominance rotation (rule)	—	+152.8%	1.07	1.29	−26.4%
Trend (rule)	—	+63.5%	0.59	0.72	−28.6%
Volatility breakout (rule)	—	+33.1%	0.42	0.42	−23.7%
Mamba (state-space)	0.526	+20.7%	0.35	0.31	−38.5%
TCN (temporal conv.)	0.516	+7.4%	0.13	0.11	−22.8%
BTC buy-and-hold	—	+8.1%	0.08	0.11	−50.1%
S&P 500 buy-and-hold	—	+46.9%	1.16	0.25	−18.8%

Three observations follow. First, the agents' classification AUCs are *modest* — between 0.50 and 0.54 — and yet several agents post large returns. This gap is itself a finding: at this horizon the realised profit is a mixture of a weak directional signal and the asymmetric ATR-bracket risk management, not pure forecasting accuracy. We are careful throughout

not to claim that a high return proves a high hit-rate. Second, the learned agents do not all survive an honest, leak-free evaluation equally: LightGBM and PatchTST remain strongly profitable, the Mamba agent is positive but volatile, and the TCN's apparent standalone edge largely evaporates once its trading parameters are chosen without peeking at the test window (+7.4%, Sharpe 0.13). Third, the dominance-rotation rule posts by far the highest single return (+152.8%), but — as discussed below — this is convex bracket behaviour rather than demonstrated timing skill, so the agent is kept as a diversifier, not an alpha source.

The LightGBM grid search. The gradient-boosting agent illustrates the value of searching jointly over model and execution parameters. For each of the 48 model configurations, the trading grid of several thousand execution variants is evaluated and the best trading configuration retained; Figure 6.1 shows how six performance metrics are distributed across those 48 models. The Sharpe and total-return histograms are centred well inside profitable territory (median Sharpe around 1.1, median return around +60%), indicating that the edge is broad over the model space rather than a single lucky point, while the OOS AUC stays tightly clustered near 0.535. The best model reaches a Sharpe of 1.77 at a +84% return. A practical advantage of gradient boosting is that, once each model is trained, evaluating thousands of trading configurations is nearly instantaneous, so the whole search completes in minutes on a laptop.

Methods that did not make the final system. Several additional paradigms were prototyped and honestly rejected. A **deep reinforcement-learning** agent (PPO over a long/flat/short action space) never developed a positive edge at the hourly horizon: its per-step reward is dominated by noise, and the policy converged only on strongly trending folds while losing through choppy ones. A **genetic-programming** agent evolved interpretable symbolic rules whose in-sample fitness did not transfer out of sample. Among the rule agents, a **contrarian sentiment** (Fear & Greed) agent returned −29.9% and fell to the 7th percentile of its random-bracket null, and a **mean-reversion** agent produced negative OOS returns; both were excluded. An experimental **cross-asset** agent showed weak directional evidence that was not significant against its null and was likewise dropped.

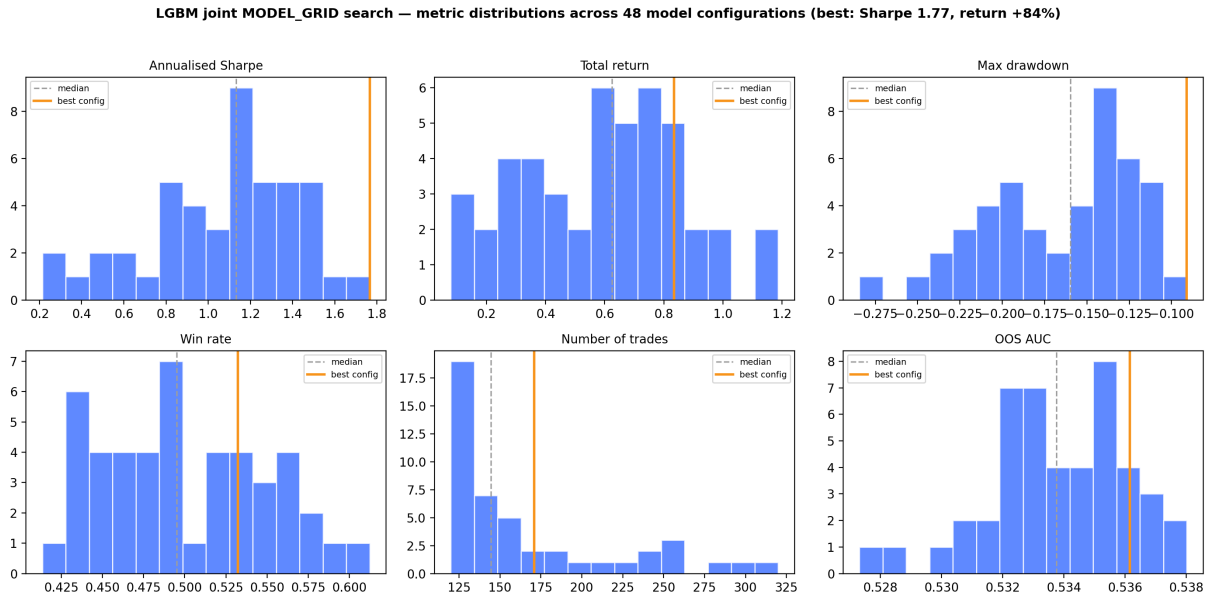

Figure 6.1: Distribution of six performance metrics across the 48 LightGBM model configurations in the joint grid search, each shown at its own best trading configuration. The dashed line marks the median; the orange line marks the best model (Sharpe 1.77, return +84%). Metrics: annualised Sharpe, total return, max drawdown, win rate, number of trades, and out-of-sample AUC.

Reporting these negatives is part of an honest account: most of what was tried did not work.

6.3 Signal Diversity

A multi-agent system can only benefit from combination if its agents are genuinely diverse. Table 6.3 reports pairwise Spearman rank correlations between the four learned agents' probability signals over the pre-OOS meta-training window. The cross-paradigm pairs are near zero, and even the TCN and PatchTST — which share an identical 32-feature diet — correlate at only 0.47, confirming that their differing inductive biases (local dilated convolution versus global patch attention) produce materially distinct rankings. The rule agents add a further layer of diversity, since their dominance-, breakout-, and trend-based signals are built from logic and, in the dominance agent's case, from an information source no price-feature model uses.

Table 6.3: Pairwise Spearman rank correlations between the four learned agents' probability signals over the meta-training window (January 2022 – 31 May 2024). Low correlations confirm genuinely diverse information sources.

	LGBM	**Mamba**	**TCN**	**PatchTST**
LGBM	1.000	0.370	0.119	−0.016
Mamba	—	1.000	0.145	0.019
TCN	—	—	1.000	0.473
PatchTST	—	—	—	1.000

6.4 The Multi-Agent Fund

The seven autonomous agents are combined into a single fund by the leak-free capped inverse-volatility allocator of Section 4.8. Table 6.4 reports the headline result against the two passive benchmarks, the naive equal-weight baseline, and the regime-gated coordinator ablation. Figure 6.2 shows the corresponding equity curves and Figure 6.3 the ranked returns and Sharpe ratios.

Table 6.4: Final multi-agent fund versus baselines and the coordinator ablation on the unified OOS window (31 May 2024 – 31 May 2026), net of fees. Alpha is measured against BTC buy-and-hold.

Strategy	**Return**	**Sharpe**	**Sortino**	**MaxDD**	α
Final MAS fund (capped inv.-vol)	**+49.7%**	**1.92**	**2.51**	**−5.3%**	+41.5 pp
Naive equal-weight fund	+54.7%	1.76	2.26	−5.2%	+46.6 pp
Coordinator ablation (Sharpe × regime)	+22.8%	0.47	0.57	−21.4%	+14.6 pp
BTC buy-and-hold	+8.1%	0.08	0.11	−50.1%	—
S&P 500 buy-and-hold	+46.9%	1.16	0.25	−18.8%	+38.8 pp

The headline is a *risk-adjusted* one. On total return alone the fund (+49.7%) is mid-pack: the strongest single agents and the equal-weight baseline (+54.7%) return more. But the fund attains the highest *Sharpe ratio* of any strategy in the study (1.92) with the smallest maximum drawdown (−5.3%), comfortably ahead of BTC buy-and-hold (Sharpe 0.08) and the S&P 500 (Sharpe 1.16). Diversifying across heterogeneous agents converts a collection of volatile individual return streams into a smooth, controlled portfolio — which is precisely what a fund-of-agents design is meant to do. The per-regime breakdown in Table 6.5 confirms that this is genuine robustness and not a single lucky sub-period: the fund is positive with a Sharpe above 1.5 in *all three* regimes, including the bear.

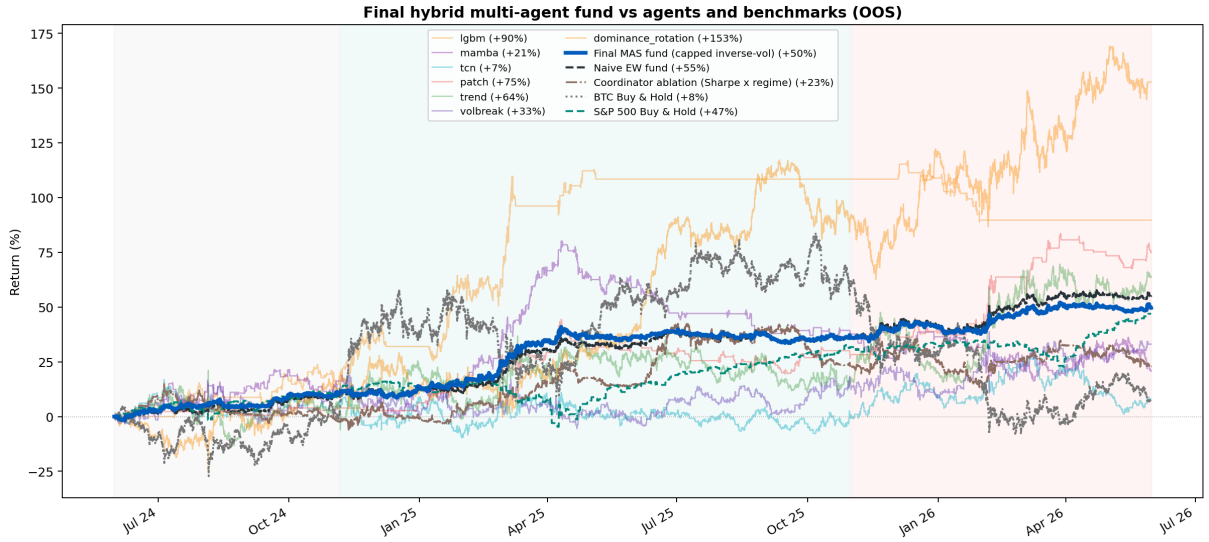

Figure 6.2: OOS equity curves of the final multi-agent fund (thick blue) against the individual agents, the two passive benchmarks, the naive equal-weight fund, and the coordinator ablation. The fund's curve is markedly smoother than any individual agent's: it gives up some of the highest agent returns in exchange for a far more controlled path (maximum drawdown −5.3% versus −50.1% for BTC).

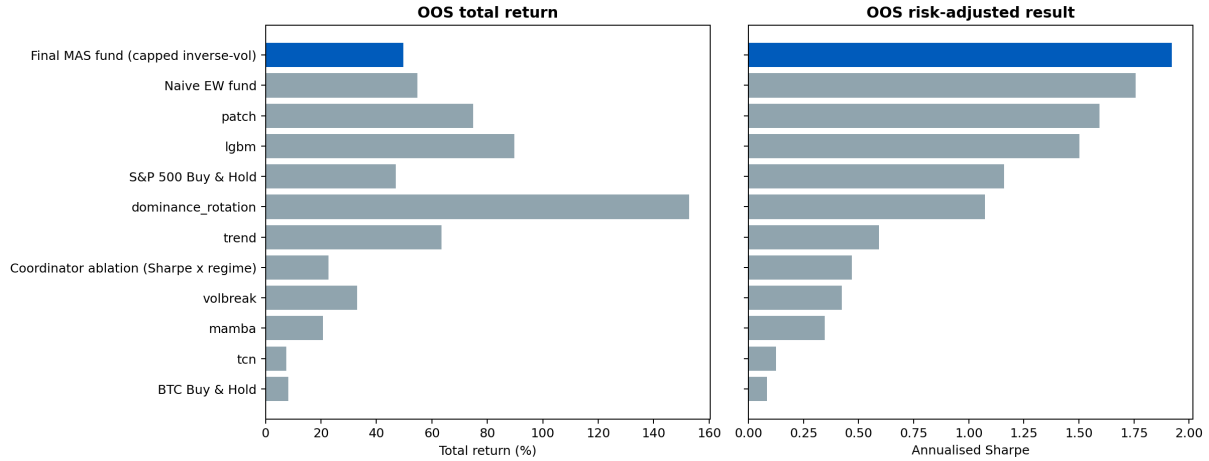

Figure 6.3: Ranked OOS total return (left) and annualised Sharpe ratio (right) for the fund, the individual agents, and the benchmarks. On total return the fund sits mid-pack — by construction, since diversification trades peak return for stability — but on risk-adjusted return it ranks first, ahead of every individual agent and both benchmarks.

Table 6.5: Final MAS fund performance decomposed by market regime (each segment rebased to its start). The fund is profitable and risk-controlled in every regime.

Regime	Return	Sharpe	MaxDD
Chop	+10.8%	2.57	−3.5%
Bull	+22.8%	1.81	−5.3%
Bear	+10.0%	1.74	−3.8%

6.4.1 Capital Allocation Over Time

Figure 6.4 shows how the capped inverse-volatility allocator distributes capital across the seven agents through the hold-out. The cap of $2/N \approx 28.6\%$ is visibly binding on

the steadiest agents (LightGBM and PatchTST), preventing either from dominating the fund, while the noisier rule agents receive smaller, time-varying weights. The mean allocations over the window are LightGBM 21.9%, PatchTST 22.1%, Mamba 14.5%, TCN 13.2%, volatility breakout 10.9%, trend 8.9%, and dominance rotation 8.6% — a genuinely distributed allocation in which no single method carries the fund. Figure 6.5 shows the month-by-month return comparison: the fund's monthly returns are visibly tighter than BTC's, trading the latter's occasional very large months (and very large losses) for consistency.

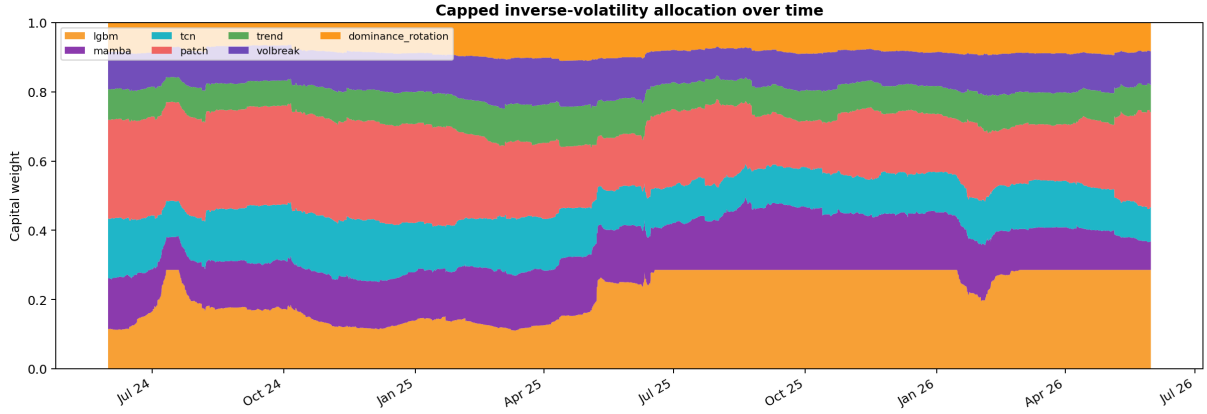

Figure 6.4: Capped inverse-volatility capital allocation over the OOS window. Each band is one agent's weight; the cap of $\approx 28.6\%$ binds on the steadiest agents (LightGBM, PatchTST) and the allocation shifts smoothly as agents' trailing volatilities change. No look-ahead is used: every weight depends only on past returns.

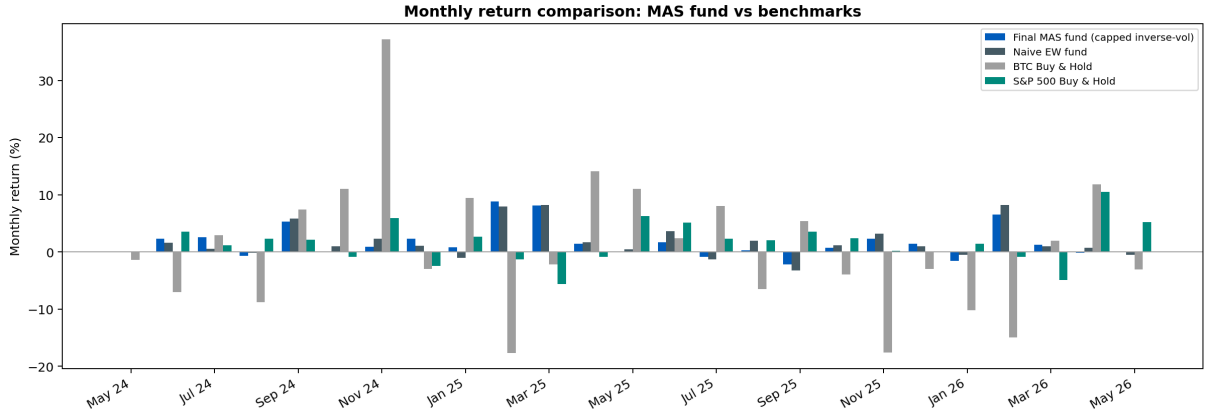

Figure 6.5: Monthly return comparison of the final fund and the naive equal-weight fund against BTC and S&P 500 buy-and-hold. The fund's monthly distribution is far tighter than BTC's, exchanging extreme months for consistency.

6.4.2 Two Honest Negative Results

Two findings deserve to be stated plainly rather than buried, because they bound what this thesis can claim.

The adaptive coordinator did not help. The regime-gated coordinator — weights set by a softmax of trailing Sharpe ratios times a per-regime competence score, the "intelligent" controller one might expect to be the centrepiece — returns only +22.8% (Sharpe 0.47, drawdown −21.4%), *worse* on every risk metric than the far simpler capped inverse-volatility rule. The lesson is sobering and worth keeping: not every component that *looks* agentic and adaptive actually improves a system. The defensible claim of this thesis is therefore hybrid multi-agent *diversification* through a transparent risk-balanced allocator, not that a learned coordinator adds alpha. The naive equal-weight fund, meanwhile, remains highly competitive and even edges the capped fund on total return; the capped allocator is preferred not because it dominates equal-weight on every metric but because it gives a more interpretable, concentration-controlled allocation while preserving most of the return and the strongest Sharpe.

A high return is not the same as alpha. The dominance-rotation agent's +152.8% is the largest single return in the study, and it is tempting to read it as a discovery. It is not. When the agent's signal is replaced by random permutations of itself fed through the same asymmetric ATR bracket, the resulting return distribution is wide, and the agent's true result sits at roughly its 95th percentile — inside, not beyond, what an essentially random signal achieves through the same convex risk management. The agent earns its place in the fund only because its return stream is structurally decorrelated from the price-based agents (it reads cross-asset dominance flow), making it a useful *diversifier*; it is explicitly not presented as proven forecasting skill. The same random-bracket discipline is what justified excluding the sentiment and cross-asset experiments. A recurring theme of this work is that, in a domain this noisy, the burden of proof on any "too good" result must be high.

These results, together with a fuller red-team critique of the system's remaining limitations, are documented in the project's internal audit notes (see Chapter 8).

7 Financial Evaluation

A predictive model with genuine directional accuracy is a necessary but not sufficient condition for a profitable trading strategy. Transaction costs, the timing and sizing of entries and exits, and the interaction between trade frequency and fee drag can turn a model with a positive AUC into an unprofitable strategy, or alternatively reveal that a strategy with a seemingly modest Sharpe ratio is already capturing a large fraction of the zero-cost theoretical maximum. This chapter formalises the financial evaluation framework used throughout the thesis and describes the multi-regime evaluation design.

7.1 Performance Metrics

All strategies are evaluated on the same held-out test window (31 May 2024 – 31 May 2026) using the following metrics.

Total return. The compounded return of a unit investment over the test period:

$$R_{\text{total}} = \prod_{i=1}^{N}(1 + r_i) - 1, \tag{7.1}$$

where r_i is the net return of the i-th trade (or hourly equity step for the continuous equity curve).

Annualised Sharpe ratio. The primary risk-adjusted performance metric, computed from the hourly equity log-return series:

$$\text{Sharpe} = \frac{\bar{\delta}}{\hat{\sigma}_\delta} \cdot \sqrt{24 \times 365}, \tag{7.2}$$

where $\bar{\delta}$ and $\hat{\sigma}_\delta$ are the sample mean and standard deviation of hourly log-returns of the equity curve. The annualisation factor $\sqrt{24 \times 365}$ reflects continuous 24/7 trading in cryptocurrency markets.

Maximum drawdown. The peak-to-trough decline of the equity curve:

$$\text{MaxDD} = \min_t \frac{\text{eq}(t) - \max_{s \leqslant t} \text{eq}(s)}{\max_{s \leqslant t} \text{eq}(s)}. \tag{7.3}$$

A lower absolute value indicates a smoother, more controlled equity curve.

Win rate. The fraction of completed trades with positive net return: $\text{WR} = \frac{1}{N} \sum_{i=1}^{N} \mathbf{1}[r_i > 0]$. Note that win rate alone is not a sufficient performance metric; a strategy can be profitable with a win rate below 50% if winning trades are larger on average than losing ones (positive reward-to-risk ratio).

Sortino ratio. A variant of the Sharpe ratio that penalises only downside deviation, making it more appropriate for strategies with positively skewed return distributions:

$$\text{Sortino} = \frac{\bar{\delta}}{\hat{\sigma}_\delta^-} \cdot \sqrt{24 \times 365}, \tag{7.4}$$

where $\hat{\sigma}_\delta^-$ is the standard deviation of hourly log-returns that are negative. A higher Sortino ratio relative to Sharpe indicates that the strategy's volatility is predominantly upside (favourable) rather than symmetric.

Alpha. Excess return relative to the passive Bitcoin buy-and-hold benchmark over the same period: $\alpha = R_{\text{strategy}} - R_{\text{B\&H}}$. A positive alpha indicates that the strategy outperformed holding the underlying asset.

Per-regime metrics. Each performance metric is also computed separately for the three market regimes defined in Table 6.1. For per-regime evaluation, the equity curve is rebased to 1.0 at the start of each regime, and buy-and-hold is similarly rebased to compute regime-specific alpha. This isolates the strategy's behaviour under each market

condition and determines whether the reported performance is attributable to specific sub-periods or reflects consistent edge across all three regimes.

7.2 Fee Model

All backtests incorporate realistic transaction costs matching the fee structure available to a retail trader on the MEXC exchange, whose published fee schedule is used throughout.[1] MEXC was selected because it offers one of the most competitive retail cost structures among major venues — notably a 0% maker fee on a wide range of markets. This is not altruism but a deliberate business model: high-volume exchanges derive the bulk of their revenue from the long tail of frequently-trading, often-overtrading participants, so keeping headline fees minimal is itself a customer-acquisition tool. At the same time, charging nothing (or very little) for resting limit orders incentivises participants to *provide* liquidity, deepening the order book and making the venue more attractive to everyone. For a systematic strategy, the practical consequence is that the maker/taker split — not the absolute fee level — becomes the dominant cost lever, which is why the strategies in this thesis are designed to maximise the fraction of passively-filled limit (maker) orders. Two types of order fill are modelled:

- **Maker orders** (0.00%): limit orders that rest on the order book and are filled passively when price returns to the limit level. A *fill buffer* of 5 basis points is applied when checking whether a limit was touched within a candle: a long limit at price L is considered filled if the candle low $\leqslant L + \epsilon$, a short limit if the candle high $\geqslant L - \epsilon$, where $\epsilon = 0.0005$. If the limit is not touched, the order expires and no trade is entered.

- **Taker orders** (0.05%): market orders that execute immediately at the current price, crossing the spread and removing liquidity.

Stop-loss exits and timeout exits are modelled as taker orders. Take-profit exits are modelled as maker orders (the TP level acts as a resting limit). Short positions accrue a funding credit of +0.00077% per hour, reflecting the historically positive funding rate

[1] `https://www.mexc.com/fee`

on BTC perpetual futures. Leverage does not alter these per-trade fee rates: fees are charged on notional position size, so the relative cost structure is invariant to the leverage multiplier; what dominates real-world execution cost beyond fees is *slippage*, which the resting-limit-order design is intended to minimise.

In the final configuration adopted for all agents, both long and short entries are placed as limit orders, filling at the maker rate when the limit is touched and reverting to taker only when the order must chase the market. Table 7.1 summarises the fee model.

Table 7.1: Fee model applied to every agent in the long+short backtest. "Maker" = 0.00%, "Taker" = 0.05%, "Funding" = +0.00077%/h received on short positions. Entries and take-profits are resting limit orders (maker if filled, taker if missed); stop-loss and timeout exits cross the spread (taker).

Side	**Entry**	**SL exit**	**TP exit**	**Timeout**	**Funding**
Long	Maker/Taker	Spot taker	Maker	Spot taker	—
Short	Maker/Taker	Fut. taker	Maker	Fut. taker	+0.00077%/h

7.3 Strategy Comparison

Table 7.2 presents the side-by-side comparison of the final multi-agent fund against its baselines and benchmarks on the unified window, all re-evaluated on the identical strict window so that the figures are directly comparable. The BTC buy-and-hold benchmark returned +8.1% over the period; alpha is measured against it. Returns are net of the fee model of Section 7.2.

Table 7.2: Financial comparison on the unified two-year OOS window (31 May 2024 – 31 May 2026), net of fees. The fund attains the strongest risk-adjusted profile — the highest Sharpe and Sortino and the smallest drawdown — of any strategy, including both passive benchmarks.

Strategy	**Return**	**Sharpe**	**Sortino**	**MaxDD**	α
BTC Buy & Hold	+8.1%	0.08	0.11	−50.1%	—
S&P 500 Buy & Hold	+46.9%	1.16	0.25	−18.8%	+38.8 pp
Naive equal-weight fund	+54.7%	1.76	2.26	−5.2%	+46.6 pp
Coordinator ablation	+22.8%	0.47	0.57	−21.4%	+14.6 pp
Final MAS fund	+49.7%	1.92	2.51	−5.3%	+41.5 pp

The financial picture confirms the conclusion of the experimental chapter. The fund does not chase the highest headline return — the naive equal-weight baseline and the strongest individual agents return more in absolute terms — but it delivers the best *risk-adjusted* profile in the study: a Sharpe of 1.92 and a maximum drawdown of just −5.3%, against a Bitcoin benchmark that returned only +8.1% while suffering a −50% drawdown, and a broad-market S&P 500 benchmark with a materially lower Sharpe. For a capital allocator, this is the relevant objective: a smoother equity curve at a comparable return is strictly preferable, because it can be levered or sized to taste while a high-drawdown curve cannot.

Per-regime evaluation objectives. Beyond the full-period aggregate, each strategy is assessed against the three market regimes of Table 6.1. The decomposition tests specific properties:

Regime 1 (Chop): A well-designed strategy should remain approximately flat or slightly positive, avoiding over-trading and fee drag in a low-signal environment. The fund returns +10.8% here at a Sharpe of 2.57, its best regime — the diversification damps the noise that hurts individual agents.

Regime 2 (Bull): The key metric is alpha versus buy-and-hold during a parabolic uptrend. The fund captures +22.8% of the run while keeping its drawdown to −5.3%, participating in the trend without taking the benchmark's full risk.

Regime 3 (Bear): This is the most decisive discriminator between strategies with and without a genuine short-side capability. Over a market that fell more than 50% from its high, the fund still returns +10.0% (Sharpe 1.74), demonstrating that its agents collectively profit or preserve capital on the short side rather than merely surviving.

That the fund is positive with a Sharpe above 1.5 in *all three* regimes is the clearest evidence that its performance is regime-robustness rather than a single directional bet.

7.4 Discussion of Fee Impact and Strategy Design

The fee results make plain a design principle that academic strategy evaluation often underweights: the fee model should be fixed as a constraint, not treated as a post-hoc adjustment. Across the agents the gap between the zero-fee and with-fee equity curves

is substantial, and it is driven almost entirely by trade frequency interacting with the maker/taker split. Two levers reduce this drag without abandoning activity: raising the trade-quality bar (higher thresholds, fewer but more confident entries) and maximising the fraction of passively-filled maker orders (limit entries and limit take-profits). The final configurations use both — which is why the strategies are deliberately built around resting limit orders rather than market orders.

An honest caveat must accompany this. Even after the per-agent position caps, the system as a whole *trades a great deal*: several agents turn over frequently, and the fund inherits their combined activity. On a venue with the assumed near-zero maker fees and reliable passive fills, this is sustainable; on a busier or less liquidity-friendly platform, the same turnover could erode much of the apparent edge. The backtest assumes that limit entries and take-profits are filled passively the great majority of the time, which is optimistic: real execution depends on queue position, spread, slippage, and changing funding rates, none of which a one-hour backtest can fully capture. The results should therefore be read as an optimistic research backtest, not a production execution simulation; execution cost is one of the dominant remaining uncertainties, and is revisited in the limitations of Chapter 8.

It is worth being candid about *why* a venue such as MEXC offers near-zero maker fees in the first place. Generating trading volume on these markets is exactly what the platforms want: low headline fees attract the long tail of frequently-trading, often-overtrading retail participants, and the resting limit orders that minimal maker fees encourage deepen the order book and make the venue more attractive still. The uncomfortable reality of this ecosystem is that the great majority of such participants lose money over time, while the exchanges earn substantial revenue from their collective activity. Cryptocurrency in particular has become a place where many people chase fast — and sometimes dirty — money, frequently with leverage they do not understand. Smaller venues like MEXC are generally less liquid, less regulated, and less safe than the largest exchanges or than the regulated instruments through which Bitcoin can also be traded. A systematic, risk-managed strategy of the kind developed here is, in part, an attempt to approach this environment with discipline rather than speculation — but the same environment is the reason its real-world execution must be treated with caution.

8 Discussion

8.1 What the Fund Does, and What It Does Not

The clearest reading of the results is that the value of this system lies in *diversification across heterogeneous agents*, realised through a transparent risk-balanced allocator, rather than in any single model or in a sophisticated learned controller. The final fund's defining property is not its return — the equal-weight baseline and the strongest individual agents return more — but its *risk-adjusted* profile: the highest Sharpe in the study at the smallest drawdown, positive in all three market regimes. A collection of volatile, individually unreliable return streams is converted into a smooth portfolio, which is exactly what a fund-of-agents is meant to achieve.

The individual agents form a clear hierarchy once they are judged under honest, leak-free selection. The LightGBM agent is the most reliable single performer: gradient boosting suits the tabular, high-dimensional feature space, and its joint feature-selection and model–trading grid search produces an execution policy well matched to its own probability distribution. The PatchTST agent is a close second, its calibrated probabilities supporting selective, low-turnover entries with a controlled drawdown. The Mamba agent is a genuine but volatile chop-specialist, and the TCN — which appears strong when its parameters are tuned on the test window — retains little standalone edge once that look-ahead is removed. This last point is itself instructive: it is a reminder of how easily an optimistic evaluation protocol can manufacture an edge that does not exist.

8.2 The Role of Feature and Model Diversity

A deliberate design decision throughout is that each agent consumes a *different* feature subset matched to its learning mechanism. This is validated by the low Spearman correlations between the learned agents' probability series (Table 6.3): the cross-paradigm pairs are near zero, and even the TCN and PatchTST — which share an identical feature diet — correlate at only 0.47, their differing inductive biases producing materially distinct rankings. The rule agents extend this diversity further, since their signals come from fixed economic logic and, for the dominance agent, from a cross-asset information source that no price-feature model touches.

From the perspective of ensemble theory, uncorrelated errors are precisely what allow a combination to reduce variance without sacrificing expected return [26]. Diversity, however, is a *necessary but not sufficient* condition: it is what makes the risk-parity allocator effective, but it does not by itself guarantee that a more elaborate combination rule will help — as the coordinator ablation shows.

8.3 Why the Adaptive Coordinator Did Not Help

The most interesting negative result of this work concerns the integration layer. The intuition behind a hybrid multi-agent system is that an intelligent controller should learn *when to trust which agent* and so beat any mechanical combination. The regime-gated coordinator was built to do exactly this — weighting agents by a softmax of their trailing Sharpe ratios times a per-regime competence score — and it nonetheless underperformed the far simpler capped inverse-volatility rule on every risk metric (+22.8% at Sharpe 0.47 versus +49.7% at Sharpe 1.92).

The reason is that adaptivity buys little when the underlying estimates are noisy. With only a two-year hold-out and a handful of regime transitions, trailing-Sharpe and competence estimates are themselves high-variance; weighting agents by them adds estimation noise faster than it adds skill, and concentrates capital into whichever agent was recently lucky. The simple inverse-volatility rule, by contrast, asks only for a stable quantity — each agent's recent volatility — and is correspondingly steady. This generalises an earlier

finding in the project's development: an even more elaborate meta-labelling supervisor, which tried to learn a profitability classifier over the agents' signals, also failed to beat its best constituent, in that case because a near-monotonic calendar feature let it memorise *when* it was trained rather than *what* works. The recurring lesson is that, in this data regime, the burden of proof on a complex coordinator is high, and simple, transparent allocation is the more defensible choice. We therefore report hybrid *diversification*, not learned coordination, as the contribution.

8.4 An Independent Red-Team Audit

Because results in this field are so easily flattered by subtle errors, the system was subjected to a deliberate adversarial review — a "red-team" audit, conducted with the help of a separate automated coding agent and recorded in the project's internal audit notes. Its conclusions are reflected throughout this thesis and are worth summarising honestly. The audit judged the current system materially more defensible than earlier versions: the learned coordinator is correctly demoted to an ablation, weak agents (sentiment, mean-reversion, cross-asset) are excluded for stated reasons, the equal-weight baseline is kept visible, and the dominance agent is described as a diversifier rather than as alpha. It also identified the limitations this thesis does not hide: the evidence is a single-asset, single two-year OOS path; the execution model is optimistic around limit fills and slippage; and much of the realised return is a mixture of weak signal, asymmetric ATR exits, and diversification rather than clean forecasting skill. Its bottom line — that the honest claim is promising research evidence gathered under limited data access, not a strategy proven ready to trade — is the claim this thesis makes.

8.5 Scope: Why Bitcoin

The system is trained and evaluated on Bitcoin alone, and this is a considered choice rather than a convenience. Forecasting BTC at an hourly horizon, with leakage controlled and costs modelled honestly, is already a hard enough problem to fill a thesis; adding assets multiplies the engineering and the ways to fool oneself without changing the core

scientific question. The broader cryptocurrency universe enters only indirectly: the other tracked coins are used to approximate a market-wide capitalisation, from which Bitcoin dominance — an input to the dominance-rotation agent — is derived. With the exception of a few BTC-specific features (most notably the halving-cycle encoding), the entire pipeline could in large part be replicated for other continuously traded crypto assets. Their higher volatility would likely make the forecasting harder and the execution costlier, but for the same reason it could also offer larger returns; this is a natural extension rather than a limitation of the present design.

8.6 Limitations

Single asset, single venue, single path. All models are trained and evaluated on BTC/USDT and costed on one exchange's fee schedule. The out-of-sample window, though deliberately multi-regime, is a single contiguous two-year path through one asset's history. This is sufficient to demonstrate architectural feasibility and promising evidence, but not to establish robustness across assets, venues, and longer macro cycles.

Execution realism. As discussed in Chapter 7, the backtest assumes favourable passive limit fills and near-zero maker fees, and the fund's aggregate turnover is high despite per-agent position caps. Stricter fill and slippage assumptions would absorb part of the apparent edge; with only two years of data, such sensitivity tests are inherently noisy. A stronger validation would need historical order-book or sub-hourly data, which was beyond the budget available for this thesis.

Signal strength versus risk management. The agents' classification AUCs are modest (roughly 0.50–0.54). The returns should therefore be understood as a combination of weak directional signal and effective ATR-bracket risk management, not as evidence of strong predictive accuracy. The random-bracket null test is included precisely to keep this distinction explicit.

Reproducibility of the deep agents. The deep sequence models do not yet fix every stochastic component (notably the Mamba training loop and `DataLoader` shuffling), so

their walk-forward outputs vary between runs and the fund inherits this non-determinism. Full bitwise reproducibility is a prerequisite for treating any single agent's contribution as fixed and is treated as a blocking task in the project's remediation notes.

Computational budget. Training was performed on a personal laptop and a single cloud A100 GPU, so the deep agents ran under modest epoch and fold budgets. The reported results should be read as a lower bound on what the same architectures could achieve with larger budgets.

8.7 Practical Usability

Of the components, the LightGBM agent is the most deployment-ready: it trains in minutes, produces usable probability estimates, and has a well-understood failure mode (probability compression toward 0.5 in low-predictability regimes). The fund layer adds little latency, since the allocation is a simple causal computation over trailing returns. The principal obstacle to live deployment is not the models but the execution assumptions: moving from this backtest to paper trading would expose the strategy to the real slippage, latency, and partial fills that — as argued throughout — ultimately matter more than headline fees.

9 Concluding Remarks

This thesis set out to design and evaluate a *hybrid multi-agent* trading system for cryptocurrency markets, built from heterogeneous artificial-intelligence paradigms, and to answer a single guiding question: can combining structurally different methods produce more stable and reliable trading decisions than any individual method on its own? Seven autonomous agents were developed — four learned models (a gradient-boosting model, a temporal convolutional network, a selective state-space model, and a patch-based Transformer) and three rule-based agents (trend following, volatility breakout, and cross-asset dominance rotation) — each consuming a feature subset matched to its inductive bias and executing through a shared, realistic ATR-bracket engine. All were evaluated under a strict, leak-free walk-forward protocol on a two-year out-of-sample window spanning a choppy range, a parabolic bull market, and a macro capitulation.

The answer is a qualified "yes," with the emphasis on *how* the combination is performed. Pooled into a fund by a transparent, leak-free capped inverse-volatility allocator, the heterogeneous agents produced the strongest risk-adjusted result in the study: a +49.7% return at an annualised Sharpe of 1.92 and a maximum drawdown of only −5.3%, profitable in all three market regimes. This sits comfortably ahead of both a Bitcoin buy-and-hold benchmark (Sharpe 0.08, drawdown −50%) and a broad-market S&P 500 benchmark (Sharpe 1.16). The fund does not maximise headline return; it converts a set of volatile, individually unreliable agents into a smooth portfolio that stays steady across regimes. That, rather than the sophistication of any one model, is the contribution.

Equally important is what *did not* work, reported here as honestly as what did. A learned, regime-gated coordinator, the kind of adaptive controller one expects to be the centrepiece of an "intelligent" system, failed to beat the far simpler risk-parity rule, because in a two-

year sample its adaptive estimates are too noisy to add skill. An earlier meta-labelling supervisor failed for a related reason. The dominance-rotation agent posts the highest single return in the study, yet a random-bracket null test shows that return to be convex risk-management behaviour rather than forecasting skill, so it is kept only as a diversifier. And the great majority of paradigms that were tried, among them deep reinforcement learning, genetic programming, a contrarian sentiment agent, a mean-reversion agent, and a cross-asset experiment, did not survive an honest out-of-sample evaluation at all. These attempts are not failures to be hidden; they are documented in full in the project repository and are part of the scientific record of the work.

If there is one lesson that this project impressed on its author more than any other, it is how genuinely *difficult* quantitative trading is as a discipline. It sits at the intersection of statistics, machine learning, software engineering, market microstructure, and financial economics, and it punishes optimism at every turn. Feature engineering was a microcosm of this. The one-hour data pipeline is limited, yet from it several hundred candidate features could be constructed, on the order of 285 in the final pool, of which only a small minority carried any stable predictive content. It was only after moving well beyond the handful of indicators derivable from raw OHLCV, and then applying the disciplined, advanced feature-selection stage recommended by the thesis supervisor, that it became possible to train anything meaningful. Most dangerous of all were features that quietly looked into the future: among the original candidates were a few that leaked forward information and, at the midpoint of the project, produced strikingly promising but entirely false results. Every feature was ultimately audited to guarantee that the final numbers are reliable. The breadth of the problem had a compensating reward, however: it forced a hands-on exploration of a wide range of state-of-the-art time-series methods, which is itself one of the lasting benefits of the work. In this sense the thesis brought together, and built upon, the years of study on the Computer Science and Intelligent Systems programme with its specialisation in Artificial Intelligence and Data Analysis.

The limitations that bound these conclusions are stated plainly in Chapter 8 and were independently scrutinised in an adversarial red-team review (its conclusions, produced with the help of a separate automated coding agent, are recorded in the project's internal

audit notes). In short: the evidence is a single-asset, single two-year path; the execution model is optimistic about passive fills and slippage, and the system trades heavily despite per-agent caps; and the agents' directional accuracy is modest, so the returns are a mixture of weak signal and effective risk management. The honest framing is that this is promising research evidence gathered under limited data access, not a strategy proven ready to trade live.

Several directions follow naturally from the findings:

- **Other assets.** The pipeline could, in large part, be replicated for other continuously traded crypto assets and beyond. Their higher volatility would make forecasting harder and execution costlier, but could equally offer larger returns; the other coins entered this thesis only to approximate the market-wide capitalisation used for Bitcoin dominance.
- **More timeframes.** Extending beyond the single one-hour resolution — combining higher- and lower-frequency views — would expose structure that an hourly pipeline cannot, at the cost of additional data and engineering.
- **More realistic execution.** The backtest assumes that a trade is almost always entered after the current candle closes, and models stop-loss triggers by a fixed intrabar rule. Relaxing both — intrabar entries, queue-aware limit fills, and more faithful stop-loss dynamics — would make the evaluation far stricter and is the single most valuable next step.
- **Broader validation.** Multiple non-overlapping out-of-sample windows, older history, and ideally order-book or sub-hourly data would turn promising evidence into a robustness claim.

For all these caveats, the central empirical observation is encouraging and, within the evaluated period, clear: the agents do carry an edge, and over a long enough horizon and a statistically meaningful number of trades, the diversified fund's profit is real and its risk well controlled. The deeper contribution is methodological: a demonstration that, in a

domain this noisy, the value of a multi-agent system lies less in the cleverness of any single model than in the *discipline of their combination* and the honesty of their evaluation.

Source code. The complete implementation — data pipeline, agents, allocator, and experiments — is maintained as a reproducible repository at `https://github.com/WojciechNeuman/hybrid-multi-agent-trading-system`. The repository is kept private; interested readers may request access by contacting the author at `wojciech.neuman@gmail.com`.

REFERENCES

[1] I. K. Nti, A. F. Adekoya, and B. A. Weyori, "A systematic review of fundamental and technical analysis of stock market predictions," *Artificial Intelligence Review*, vol. 53, no. 4, pp. 3007–3057, 2020.

[2] J. D. Hamilton, *Time series analysis*. Princeton university press, 2020.

[3] P. Bangert, *Machine learning and data science in the oil and gas industry: Best practices, tools, and case studies*. Gulf Professional Publishing, 2021.

[4] C. Zhang, N. N. A. Sjarif, and R. Ibrahim, "Deep learning models for price forecasting of financial time series: A review of recent advancements: 2020–2022," *Wiley Interdisciplinary Reviews: Data Mining and Knowledge Discovery*, vol. 14, no. 1, p. e1519, 2024.

[5] A. Tsantekidis, N. Passalis, A. Tefas, J. Kanniainen, M. Gabbouj, and A. Iosifidis, "Using deep learning for price prediction by exploiting stationary limit order book features," *Applied Soft Computing*, vol. 93, p. 106401, 2020.

[6] M. Sewell, "Characterization of financial time series," *Rn*, vol. 11, no. 01, p. 01, 2011.

[7] G. Sonkavde, D. S. Dharrao, A. M. Bongale, S. T. Deokate, D. Doreswamy, and S. K. Bhat, "Forecasting stock market prices using machine learning and deep learning models: A systematic review, performance analysis and discussion of implications," *International Journal of Financial Studies*, vol. 11, no. 3, p. 94, 2023.

[8] J. R. M. Hosking, "Fractional differencing," *Biometrika*, vol. 68, no. 1, pp. 165–176, 1981.

[9] M. L. de Prado, *Advances in Financial Machine Learning*. Wiley, 2018.

[10] O. B. Sezer, M. U. Gudelek, and A. M. Ozbayoglu, "Financial time series forecasting with deep learning: A systematic literature review: 2005–2019," *Applied soft computing*, vol. 90, p. 106181, 2020.

[11] G. Ke, Q. Meng, T. Finley, T. Wang, W. Chen, W. Ma, Q. Ye, and T.-Y. Liu, "Lightgbm: A highly efficient gradient boosting decision tree," in *Advances in Neural Information Processing Systems*, vol. 30, 2017.

[12] S. Bai, J. Z. Kolter, and V. Koltun, "An empirical evaluation of generic convolutional and recurrent networks for sequence modeling," *arXiv preprint arXiv:1803.01271*, 2018. [Online]. Available: https://arxiv.org/abs/1803.01271

[13] A. van den Oord, S. Dieleman, H. Zen, K. Simonyan, O. Vinyals, A. Graves, N. Kalchbrenner, A. Senior, and K. Kavukcuoglu, "WaveNet: A generative model for raw audio," *arXiv preprint arXiv:1609.03499*, 2016.

[14] T. Salimans and D. P. Kingma, "Weight normalization: A simple reparameterization to accelerate training of deep neural networks," in *Advances in Neural Information Processing Systems*, vol. 29, 2016.

[15] M. Wooldridge and N. R. Jennings, "Intelligent agents: Theory and practice," *The knowledge engineering review*, vol. 10, no. 2, pp. 115–152, 1995.

[16] I. Guyon and A. Elisseeff, "An introduction to variable and feature selection," *Journal of Machine Learning Research*, vol. 3, pp. 1157–1182, 2003.

[17] M. A. Hall, "Correlation-based feature selection for machine learning," 1999, PhD thesis.

[18] H. Peng, F. Long, and C. Ding, "Feature selection based on mutual information: Criteria of max-dependency, max-relevance, and min-redundancy," *IEEE Transactions on Pattern Analysis and Machine Intelligence*, vol. 27, no. 8, pp. 1226–1238, 2005.

[19] T. Fawcett, "An introduction to ROC analysis," *Pattern Recognition Letters*, vol. 27, no. 8, pp. 861–874, 2006.

[20] M. B. Kursa and W. R. Rudnicki, "Feature selection with the Boruta package," *Journal of Statistical Software*, vol. 36, no. 11, pp. 1–13, 2010.

[21] A. Bommert, X. Sun, B. Bischl, J. Rahnenführer, and M. Lang, "Benchmark for filter methods for feature selection in high-dimensional classification data," *Computational Statistics & Data Analysis*, vol. 143, p. 106839, 2020.

[22] I. Loshchilov and F. Hutter, "Decoupled weight decay regularization," *arXiv preprint arXiv:1711.05101*, 2019.

[23] A. Gu and T. Dao, "Mamba: Linear-time sequence modeling with selective state spaces," *arXiv preprint arXiv:2312.00752*, 2023.

[24] Y. Nie, N. H. Nguyen, P. Sinthong, and J. Kalagnanam, "A time series is worth 64 words: Long-term forecasting with transformers," *arXiv preprint arXiv:2211.14730*, 2023.

[25] C. Guo, G. Pleiss, Y. Sun, and K. Q. Weinberger, "On calibration of modern neural networks," in *Proceedings of the 34th International Conference on Machine Learning (ICML)*, 2017, pp. 1321–1330.

[26] T. G. Dietterich, "Ensemble methods in machine learning," in *Multiple Classifier Systems*. Springer, 2000, pp. 1–15.

Glossary

This glossary collects the acronyms and domain-specific terms used in the thesis, each with a short description. Where an acronym first appears in the text it is hyperlinked to this list. The glossary is separate from the bibliographic references at the end of the document.

Acronyms

Acronym	Expansion	Description
AI	Artificial Intelligence	Computational methods that learn or reason from data.
ADF	Augmented Dickey–Fuller	Statistical test for whether a series is stationary (has no unit root).
ATR	Average True Range	A measure of recent price volatility, used to size stops and targets.
AUC	Area Under the ROC Curve	Ranking quality of a classifier; 0.5 is random, 1.0 is perfect.
B&H	Buy-and-Hold	Passive benchmark that buys the asset once and holds it.
BTC	Bitcoin	The cryptocurrency studied throughout the thesis.
CMF	Chaikin Money Flow	Volume-weighted momentum oscillator.
EMA	Exponential Moving Average	Moving average that weights recent prices more heavily.
ETH	Ethereum	Second-largest cryptocurrency; used for the ETH/BTC cross.
GRU	Gated Recurrent Unit	A recurrent-network cell; prototyped but not retained.
LGBM	LightGBM	Gradient-boosted decision-tree framework; the tabular agent.

Acronym	Expansion	Description
LOB	Limit Order Book	The full set of resting buy and sell orders at each price.
LSTM	Long Short-Term Memory	A recurrent-network cell with gating for long sequences.
MACD	Moving Average Convergence Divergence	Momentum indicator from the gap between a fast and slow EMA.
MAS	Multi-Agent System	A set of autonomous agents combined into one decision-maker.
MaxDD	Maximum Drawdown	Largest peak-to-trough loss of an equity curve.
MLP	Multi-Layer Perceptron	A basic feed-forward neural network.
MPS	Metal Performance Shaders	Apple's GPU backend, used for local model training.
OHLCV	Open, High, Low, Close, Volume	The five fields of a price candle.
OOS	Out-Of-Sample	Data held out from training, used for honest evaluation.
PPO	Proximal Policy Optimisation	A reinforcement-learning algorithm; prototyped, not retained.
RL	Reinforcement Learning	Learning to act by maximising cumulative reward.
RSI	Relative Strength Index	Momentum oscillator bounded between 0 and 100.
SMA	Simple Moving Average	Unweighted average of recent prices.
SSM	State-Space Model	Sequence model with a learned latent state (the Mamba agent).
TBM	Triple Barrier Method	Labelling that classifies a trade by which of three barriers it hits first.
TCN	Temporal Convolutional Network	Sequence model built from dilated causal convolutions.
TFI	Taker Flow Imbalance	Net aggressive buy-versus-sell pressure from taker volume.
VWAP	Volume-Weighted Average Price	Average traded price weighted by volume.

Acronym	Expansion	Description
WFO	Walk-Forward Optimisation	Repeatedly train on the past and test on the next period.

Key terms

Term	Description
Heterogeneous	Components built from categorically different learning paradigms, each with its own bias and failure mode.
Hybrid	The integration of dissimilar components into a single decision-making system.
Fund-of-agents	The framing of the system as a portfolio of autonomous sub-strategies combined by a capital allocator.
Inverse-volatility risk parity	Allocation rule that weights each agent inversely to its trailing return volatility, equalising risk contributions.
Random-bracket null	Significance test that re-runs an agent's signal as random permutations through the same bracket, to separate timing skill from risk-management effects.
Meta-labelling	A two-stage scheme in which a second model predicts whether a primary trading signal will be profitable.
Fractional differentiation	A transform that makes a price series stationary while keeping long-memory content, set by a fractional order d.
Sharpe ratio	Mean excess return divided by its standard deviation, annualised; the main risk-adjusted metric.
Sortino ratio	A Sharpe variant that penalises only downside volatility.
Alpha	Return in excess of a passive benchmark over the same period.
Regime	A persistent market condition (here: chop, bull, or bear) with characteristic trend and volatility.